

Arrays & Strings

అన్ని patterns కీ పునాది. Array manipulation, prefix sums, string handling – ఇక్కడే మొదలు.

ప్రతి problem కి: ఏ pattern ఇది → ఎందుకు ఆ pattern → dry run → optimal JavaScript code → complexity → edge cases. [DSA_Patterns_Telugu.pdf](#)
pattern-first దృష్టి; ఈ file ఆ patterns ని నిజమైన LeetCode problems మీద అమలు చేస్తుంది.

ఈ document చదివిన తర్వాత Array & String problems మళ్ళీ నిన్ను భయపెట్టవు. ప్రతి problem కి **ఎలా ఆలోచించాలి** (naive నుండి insight వరకు), ఏ **pattern** వాడాలి, real-life analogy, clean JavaScript solution, dry run, complexity, గుర్తుంచుకోవాల్సినది, మరియు సాధారణ తప్పులు – అన్నీ ఉంటాయి. లక్ష్యం: "ఒకసారి చదివితే మర్చిపోకూడదు."

ముఖ్య గమనిక: నీకు DSA అస్సలు తెలియకపోతే, ముందు [DSA_00_Foundations_Telugu.md](#) చదువు – అందులో array అంటే memory లో ఎలా ఉంటుంది, Big-O అంటే ఏమిటి, time/space complexity, two-pointer/hashing basics ఉన్నాయి. ఇక్కడ మనం నేరుగా LeetCode Top-Interview-150 లోని **Array / String** section (24 problems) ని senior (SSE) interview స్థాయిలో పట్టేస్తాం. Solutions అన్నీ **JavaScript (ES2020+)** లో.

విషయ సూచిక (Table of Contents)

మొదట Patterns నేర్చుకో (ఇవి 24 problems కి పునాది):

- Pattern: Prefix / Suffix Products
- Pattern: In-place Array Manipulation (Read/Write pointers)
- Pattern: Greedy on Arrays
- Pattern: Two-pass (Left-to-right + Right-to-left)
- Pattern: String Parsing

Problems (LeetCode Top Interview 150 – Array / String):

1. Merge Sorted Array (#88) – Easy
2. Remove Element (#27) – Easy
3. Remove Duplicates from Sorted Array (#26) – Easy
4. Remove Duplicates from Sorted Array II (#80) – Medium
5. Majority Element (#169) – Easy
6. Rotate Array (#189) – Medium
7. Best Time to Buy and Sell Stock (#121) – Easy
8. Best Time to Buy and Sell Stock II (#122) – Medium
9. Jump Game (#55) – Medium
10. Jump Game II (#45) – Medium
11. H-Index (#274) – Medium
12. Insert Delete GetRandom O(1) (#380) – Medium
13. Product of Array Except Self (#238) – Medium
14. Gas Station (#134) – Medium
15. Candy (#135) – Hard
16. Trapping Rain Water (#42) – Hard

17. Roman to Integer (#13) – Easy
18. Integer to Roman (#12) – Medium
19. Length of Last Word (#58) – Easy
20. Longest Common Prefix (#14) – Easy
21. Reverse Words in a String (#151) – Medium
22. Zigzag Conversion (#6) – Medium
23. Find the Index of the First Occurrence in a String (#28) – Easy
24. Text Justification (#68) – Hard

Patterns (ముందు ఇవి పట్టుకో)

Interview లో success అంటే 200 problems బట్టి పట్టడం కాదు – **10-12 patterns** లోతుగా అర్థం చేసుకోవడం. కొత్త problem వచ్చినప్పుడు "ఇది ఏ pattern?" అని గుర్తుపడితే, solution దానంతట అదే బయటకు వస్తుంది. కింద ఈ 24 problems కి కావాల్సిన అయిదు core patterns ఉన్నాయి.

Pattern: Prefix / Suffix Products

వివరణ

Array లో ప్రతి index `i` కి, దాని **ఎడమవైపు అన్నిటి combined value** (prefix) మరియు **కుడివైపు అన్నిటి combined value** (suffix) ముందే లెక్కించి పెట్టుకోవడం. Product కావచ్చు, sum కావచ్చు, max కావచ్చు. అప్పుడు ఏ index దగ్గరైనా "నా ఎడమవైపు + కుడివైపు" answer $O(1)$ లో దొరుకుతుంది – మళ్ళీ మళ్ళీ loop తిప్పాల్సిన అవసరం లేదు.

Real-life analogy: నువ్వు ఒక queue లో నిలబడ్డావు. "నా ముందు ఎంతమంది ఉన్నారు, నా వెనుక ఎంతమంది ఉన్నారు?" అని ప్రతిసారీ లెక్కపెట్టడం కష్టం. బదులుగా, queue మొదలు నుండి నడుస్తూ "ఇక్కడిదాకా ఎంతమంది" అని ఒక counter (prefix), చివర నుండి నడుస్తూ ఇంకో counter (suffix) పెట్టుకుంటే – ఏ వ్యక్తి దగ్గరైనా వెంటనే చెప్పేయవచ్చు.

PREFIX / SUFFIX PRODUCTS · నా ఎడమవైపు × నా కుడివైపు

ARRAY



PREFIX – నా ఎడమవైపు అన్నిటి లబ్ధం



SUFFIX – నా కుడివైపు అన్నిటి లబ్ధం



answer = prefix × suffix

ఎందుకు division వాడకూడదు

సున్నా ఉంటే division పగులుతుంది – ఒక సున్నా ఉంటే ఒక్క వేట తప్పు అంతా 0,
రెండు సున్నాలుంటే అంతా 0. ఈ edge cases ని handle చేయడం కంటే
prefix × suffix చాలా సుభ్రం – మరియు interviewer అడిగేది కూడా అదే.

సాధారణీకరణ: "ప్రతి స్థానానికి – నన్ను మినహాయించి మిగతా అన్నిటి మీద ఒక operation" అనే ఏ problem కైనా ఇదే నమూనా.
Sum, product, max – operation మారుతుంది, నిర్మాణం మారదు.

ఎలా గుర్తించాలి (recognition signals)

- "ప్రతి element కి, మిగతా అన్నిటి product/sum కావాలి" అని అడిగితే.
- Division వాడకూడదు, లేదా zero వల్ల division కుదరదు అన్నప్పుడు.
- Nested loop ($O(n^2)$) కనిపిస్తోంది కానీ "ఎడమ + కుడి" విడదీస్తే $O(n)$ అవుతుందా అని అనిపిస్తే.

Tiny Template

```
const prefix = new Array(n).fill(1);
for (let i = 1; i < n; i++) prefix[i] = prefix[i - 1] * nums[i - 1]; // ఎడమవైపు
product
let suffix = 1;
for (let i = n - 1; i >= 0; i--) { result[i] = prefix[i] * suffix; suffix *= nums[i];
}
```

Pattern: In-place Array Manipulation (Read/Write pointers)

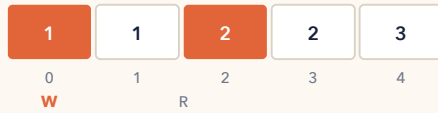
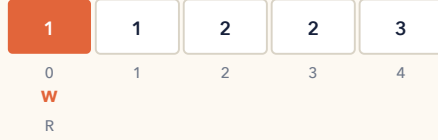
వివరణ

కొత్త array create చేయకుండా, **ఇచ్చిన array లోనే** మార్పులు చేయడం – extra space $O(1)$ లో. దీనికి కీలకం
రెండు pointers: ఒక **read** pointer (అన్ని elements ని చదువుతూ ముందుకు వెళ్తుంది), ఒక **write**
pointer (ఏవి ఉంచాలో వాటిని array మొదటికి రాస్తూ వెళ్తుంది). **write** ఎప్పుడూ **read** ని దాటదు, కాబట్టి
ఇంకా చదవని data ని overwrite చేయవు.

Real-life analogy: ఒక bookshelf లో పాడైన పుస్తకాలు తీసేసి, మంచివి మాత్రమే ఎడమవైపుకి జరపాలి. కొత్త shelf తీసుకోకుండా, ఒక చేత్తో పుస్తకాలు ఒక్కొక్కటి చూస్తూ (read), మంచిదైతే మరో చేత్తో ఎడమవైపు తర్వాతి ఖాళీలో పెడుతూ (write) వెళ్తావు. చివరికి ఎడమవైపు మంచి పుస్తకాలే మిగులుతాయి.

READ / WRITE POINTERS • IN-PLACE FILTERING

READ ముందుకి వెళ్తుంది • WRITE మంచివాటిని మాత్రమే పేర్కొనుంది



R దగ్గరికి కొత్తదా? అవును $\rightarrow \text{arr}[W++] = \text{arr}[R]$
కాదా? $\rightarrow W$ కదలదు, R మాత్రం ముందుకి



ఈ pattern యొక్క గొప్పతనం

$O(1)$ extra space – కొత్త array అవసరం లేదు.
W ఎప్పుడూ R ని దాటదు, కాబట్టి ఇంకా చదవని data ని పాడు చేయదు.
ఇదే "remove duplicates", "move zeroes", "remove element" అన్నిటికీ ఆధారం.

ఎలా గుర్తించాలి

- "In-place గా చెయ్యి", "extra array వాడకు", " $O(1)$ space" అని constraint లో ఉంటే.
- "Return the new length" లేదా "modify nums in place" అని అడిగితే (LeetCode array problems లో classic).
- Elements తీసేయడం, duplicates తగ్గించడం, ఒకచోటికి జరపడం లాంటివి.

Tiny Template

```
let write = 0;
for (let read = 0; read < nums.length; read++) {
  if (shouldKeep(nums[read])) { // ← ఈ element ని ఉంచాలా? (నీ condition ఇక్కడ)
    nums[write] = nums[read];
    write++;
  }
}
return write; // ఇదే కొత్త length (valid elements count)
```

Pattern: Greedy on Arrays

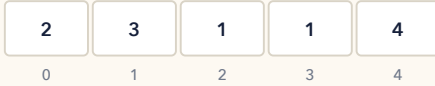
వివరణ

ప్రతి అడుగులోనూ ఆ క్షణంలో **best అనిపించే choice** తీసుకోవడం, back చూడకుండా, future ని brute-force చెయ్యకుండా. Greedy పని చేయాలంటే: local best తీసుకుంటే global best కి దారి తీస్తుందని **నిరూపించగలగాలి** (exchange argument). అన్ని problems కి పని చెయ్యదు – కానీ పని చేసినప్పుడు అద్భుతంగా simple & fast (సాధారణంగా $O(n)$).

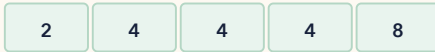
Real-life analogy: నువ్వు రోడ్డు మీద నడుస్తూ డబ్బులు ఏరుకుంటున్నావు, వెనక్కి వెళ్ళే వీలు లేదు. ప్రతి అడుగులో "ఇప్పటిదాకా నేను ఎంత దూరం వెళ్ళగలను (farthest reach)" అనేది update చేసుకుంటూ ముందుకు వెళ్ళావు. ప్రతి crossroad దగ్గర gut decision – కానీ ఆ decision correct అని logic చెప్తుంది.

GREEDY ON ARRAYS • ఒక్క సంఖ్యని మోసుకెళ్ళడం

JUMP GAME • ప్రతి index lo "నేను ఎంత దూరం వెళ్ళగలను"



ప్రతి అడుగులో $reach = \max(reach, i + \text{nums}[i])$



reach ఇలా పెరుగుతుంది →

Greedy ఎందుకు సరిపోతుంది

ఏ index ని దాటగలిగానో అది ముఖ్యం కాదు – నేను చేరగలిగే అత్యంత దూరం ఒక్కటే ముఖ్యం. అందుకే ఒక్క సంఖ్య చాలు, DP అవసరం లేదు.

ఎప్పుడు greedy fail అవుతుంది

"ఇప్పుడు ఉత్తమమైనది" ఎప్పుడూ "మొత్తానికి ఉత్తమమైనది" కాకపోతే. అప్పుడు DP కి మారాలి (Coin Change లాంటివి).

$i > reach$ అయితే – ఆ index కి ఎప్పటికీ చేరలేం → false

ఎలా గుర్తించాలి

- "Maximum/minimum ఎంత", "reach చేయగలమా", "కనిష్ట steps" – optimization problems.
- Sorting తర్వాత ఒక్క pass సరిపోతుందా అనిపిస్తే.
- DP వాడొచ్చు కానీ "ప్రతి దగ్గర best local move చాలు" అని pattern కనిపిస్తే (Jump Game, Gas Station).

Tiny Template

```
let best = 0; // farthest / maxProfit / running answer
for (let i = 0; i < n; i++) {
  best = Math.max(best, /* ఈ position వల్ల వచ్చే value */);
  // అవసరమైతే: ఇక్కడ ఒక decision తీసుకో (commit)
}
return best;
```

Pattern: Two-pass (Left-to-right + Right-to-left)

వివరణ

ఒక్క direction లో చూస్తే సరిపోని problems కి – **రెండు సార్లు** array మీద నడవడం. మొదటి pass ఎడమ నుండి కుడికి ఒక constraint satisfy చేస్తుంది; రెండో pass కుడి నుండి ఎడమకి ఇంకో constraint satisfy చేస్తూ, మొదటి పని పాడవకుండా `Math.max / Math.min` తో merge చేస్తుంది. "ఎడమ neighbor తో సంబంధం + కుడి neighbor తో సంబంధం" రెండూ కావాల్సినప్పుడు ఇది perfect.

Real-life analogy: ఒక class లో పిల్లలకు మార్కుల ప్రకారం chocolates పంచాలి – "పక్కవాడి కంటే ఎక్కువ మార్కులుంటే ఎక్కువ chocolate". ఎడమ నుండి నడిచి "ఎడమ పక్కవాడితో పోలిస్తే సరిపోయిందా" చూస్తావు. తర్వాత కుడి నుండి నడిచి "కుడి పక్కవాడితో పోలిస్తే సరిపోయిందా" చూస్తావు. రెండు rules నీ మనసులో ఒకేసారి పట్టవు – అందుకే రెండు passes.

TWO-PASS • ఎడమ నుంచి ఒకసారి, కుడి నుంచి ఒకసారి

CANDY / TRAPPING RAIN WATER సమాసా

ఒక్క pass లో ఒక వైపు సమాచారం మాత్రమే తెలుస్తుంది



ఎందుకు రెండు passes అవసరం

ఒక element యొక్క జవాబు దాని రెండు వైపులా ఉన్న సమాచారం మీద ఆధారపడితే –
ఒక్క pass లో అది సాధ్యం కాదు (భవిష్యత్తు ఇంకా తెలియదు).
రెండు passes = $O(n)$ time, $O(1)$ space. Two pointers తో $O(1)$ space కి తగ్గించవచ్చు.

ఎలా గుర్తించాలి

- ప్రతి element దాని **రెండు వైపుల** neighbors మీద ఆధారపడితే (Candy, Trapping Rain Water, Product Except Self).
- ఒక్క pass లో ఒక constraint break అవుతుంటే.
- "Left max & right max" లేదా "both sides" అనే మాట రాగానే.

Tiny Template

```
const res = new Array(n).fill(baseValue);
for (let i = 1; i < n; i++) /* ఎడమ neighbor rule */ res[i] = f(res[i - 1]);
for (let i = n - 2; i >= 0; i--) /* కుడి neighbor rule */ res[i] = Math.max(res[i],
g(res[i + 1]));
```

Pattern: String Parsing

వివరణ

String ని character by character చదువుతూ, index pointers తో manual గా tokens (words, numbers, symbols) విడదీయడం. `split / trim / regex` లాంటి built-ins convenient – కానీ interview లో "built-in వాడకుండా చెయ్యి" అంటారు, అప్పుడు index తో **whitespace skip చెయ్యడం, word boundary పట్టుకోవడం** తెలిసుండాలి. String immutable కాబట్టి, కొత్త string కట్టేటప్పుడు array లో push చేసి చివర్లో `join` చెయ్యడం efficient.

Real-life analogy: ఒక పొడవైన వాక్యాన్ని పదాలుగా విడదీయడం. కంటితో వాక్యం మీద నడుస్తూ – ఖాళీలు వస్తే skip, అక్షరం వస్తే ఆ పదం పూర్తయ్యేదాకా చదువు, ఖాళీ వచ్చాక ఒక పదం పూర్తయిందని గుర్తించు. మనిషి సహజంగా చేసేదే – దాన్ని code లో pointers తో రాస్తాం.

STRING PARSING • TOKENS ని ఏరడం

STRING PARSING • స్థితిని మోసుకెళ్ళే ఒక్క pass



రెండు సాధారణ మ్యాప్లు

1 • Split & filter

`s.split(/\s+/).filter(Boolean)`
సులభం, చదవడానికి బాగుంటుంది.
కానీ కొత్త array – $O(n)$ extra space.

2 • Manual pointer scan

`i` ని కదిలిస్తూ, spaces ని దాటుతూ,
ఒక్కే token ని చేతితో పేర్చడం.
 $O(1)$ space – interviewer ఇష్టపడతాడు.

తరచుగా మర్చిపోయే edge cases

ముందు/వెనుక spaces • వరుసగా బహుళ spaces • ఖాళీ string
ఒకే ఒక్క word • అన్ని spaces మాత్రమే
ఇవి ప్రత్యేకంగా test చేయడం – string problems lo నగ్గో bugs ఇక్కడే.

ఎలా గుర్తించాలి

- Input ఒక string, output words/tokens మీద ఆధారపడితే.
- "Reverse words", "last word length", "parse", "justify", "pattern match" లాంటివి.
- Leading/trailing/multiple spaces edge cases ఉంటే.

Tiny Template

```
let i = 0;
while (i < s.length) {
  while (i < s.length && s[i] === ' ') i++; // ఖాళీలు skip
  let start = i;
  while (i < s.length && s[i] !== ' ') i++; // ఒక word చదువు
  if (i > start) { const word = s.slice(start, i); /* వాడు */ }
}
```

Problems

1. Merge Sorted Array (LeetCode #88) – Easy

- సమస్య:** రెండు sorted integer arrays `nums1`, `nums2` ఇస్తారు. `nums1` లో మొదటి `m` elements valid, తర్వాత `n` సలాట్లు `0` (ఖాళీ). `nums2` లో `n` elements. రెండింటినీ కలిపి **ఒకే sorted array** గా `nums1` లోనే (in-place) పెట్టాలి. `nums1.length === m + n`.
- Key constraints:** Extra array వాడకుండా చెయ్యగలగాలి (interview follow-up). రెండూ already sorted (ascending).
- ఉదాహరణ:**

```
Input:  nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
Output: nums1 = [1,2,2,3,5,6]
```

- ఎలా ఆలోచించాలి:**

మొదటి ఆలోచన: `nums2` ని `nums1` చివర్లో పెట్టి, మొత్తం sort చేసేస్తే? పని చేస్తుంది కానీ $O((m+n) \log(m+n))$ – అది "already sorted" అనే బహుమతిని వృథా చేయడమే.

రెండో ఆలోచన: రెండింటినీ ముందు నుండి (index 0) పోల్చుతూ చిన్నదాన్ని తీర్చాలి. కానీ ఇక్కడ trap ఉంది – `nums1` ముందు భాగంలోనే valid data ఉంది. ముందు నుండి రాయడం మొదలుపెడితే, ఇంకా చదవని `nums1` elements ని overwrite చేసేస్తాం! కొత్త array అయితే ఫర్వాలేదు, కానీ in-place లో ఇది disaster.

ఏం గమనించాలి? `nums1` చివర్లో ఖాళీ (0 లు) ఉంది. అంటే **వెనుక నుండి** రాస్తే? అక్కడ overwrite అయ్యే ప్రమాదం లేదు – ఎందుకంటే అవి ఇప్పటికే ఖాళీ slots! ఇదీ insight: **పెద్ద elements ని వెనుక నుండి పెడుతూ రా**. రెండు arrays లోని అతిపెద్ద element ఏదో అది చివరి ఖాళీ slot కి వెళ్తుంది. ఇలా వెనక్కి వస్తే, మనం ఇంకా వాడాల్సిన `nums1` data ని ఎప్పుడూ తొక్కియం.

- Brute Force:** `nums1` లో `nums2` copy చేసి, `nums1.sort((a,b)=>a-b)`. Time $O((m+n) \log(m+n))$, Space $O(1)$ (in-place sort అయినా). సరిపోతుంది కానీ sorted అనే property ని వాడలేదు – interviewer కి నచ్చదు.
- Optimal Approach: Two pointers from the end** (In-place manipulation pattern). `i = m-1` (`nums1` valid చివరి), `j = n-1` (`nums2` చివరి), `k = m+n-1` (రాయాల్సిన చివరి slot). `nums1[i]` vs `nums2[j]` పోల్చి పెద్దదాన్ని `nums1[k]` కి పెట్టి, ఆ pointer & `k` తగ్గించు.
- Solution (JavaScript):**

```
/**
 * @param {number[]} nums1
 * @param {number} m - nums1 లో valid elements
```

```

* @param {number[]} nums2
* @param {number} n - nums2 లో elements
* @return {void} nums1 ని in-place గా modify చేస్తాం
*/
var merge = function (nums1, m, nums2, n) {
    let i = m - 1; // nums1 లోని చివరి valid element
    let j = n - 1; // nums2 లోని చివరి element
    let k = m + n - 1; // రాయాల్సిన చివరి position (ఖాళీ slot)

    // nums2 అయిపోయేదాకా నడుపు. nums2 అయిపోతే, nums1 మిగతావి already సరైన చోట ఉన్నాయి.
    while (j >= 0) {
        // nums1 లో ఇంకా elements ఉన్నాయా & అది nums2 దానికంటే పెద్దదా?
        if (i >= 0 && nums1[i] > nums2[j]) {
            nums1[k] = nums1[i]; // nums1 element ని వెనక్కి పెట్టు
            i--;
        } else {
            nums1[k] = nums2[j]; // లేదంటే nums2 element ని పెట్టు
            j--;
        }
        k--; // చివరి slot ఎప్పుడూ ఒకటి ముందుకి (ఎడమకి) జరుగుతుంది
    }
};

```

- **Dry Run:** `nums1=[1,2,3,0,0,0]`, `m=3`, `nums2=[2,5,6]`, `n=3`. మొదట `i=2`, `j=2`, `k=5`.

step	nums1[i]	nums2[j]	పెద్దది	nums1 (after)	i	j	k
1	3	6	$6 \rightarrow k=5$	[1,2,3,0,0, 6]	2	1	4
2	3	5	$5 \rightarrow k=4$	[1,2,3,0, 5 ,6]	2	0	3
3	3	2	$3 \rightarrow k=3$	[1,2,3, 3 ,5,6]	1	0	2
4	2	2	$2(\text{nums2}) \rightarrow k=2$	[1,2, 2 ,3,5,6]	1	-1	1

`j = -1` అయింది $\rightarrow$ loop ఆగుతుంది. మిగిలిన `nums1[0..1]=[1,2]` already సరైన చోట. Result `[1,2,2,3,5,6]`. 

- **Complexity:** Time $O(m + n)$ – ప్రతి element ని ఒక్కసారే touch చేస్తాం. Space $O(1)$ – extra array లేదు, `nums1` లోనే.
- **గుర్తుంచుకోవాల్సినది:** "In-place లో ముందు నుండి రాస్తే **overwrite** అవుతుందా? అయితే వెనుక నుండి రాయి." – ఇది చాలా in-place array problems కి golden rule. ఖాళీ space ఎక్కడ ఉందో అక్కడి నుండి fill చెయ్యడం మొదలుపెట్టు.
- **సాధారణ తప్పులు:**
 - ముందు నుండి merge చెయ్యడానికి ప్రయత్నించి `nums1` data ని corrupt చెయ్యడం.
 - `while (i >= 0 && j >= 0)` అని రాసి, `nums2` ఇంకా మిగిలుంటే మర్చిపోవడం. మనం `while (j >= 0)` వాడాలి కాబట్టి `nums2` elements అన్నీ కచ్చితంగా వెళ్తాయి. (`nums1` మిగిలితే ఏం చెయ్యక్కర్లేదు – అవి already చోట్లో ఉన్నాయి.)

- o `i >= 0` check మర్చిపోవడం – `nums1` valid elements అయిపోయాక `nums1[-1]` `undefined` తో పోలిక తప్పు answer ఇస్తుంది.

2. Remove Element (LeetCode #27) – Easy

- **సమస్య:** ఒక array `nums` మరియు ఒక value `val` ఇస్తారు. `nums` లోని అన్ని `val` occurrences ని **in-place** గా తీసేసి, మిగిలిన elements count `k` ని return చెయ్యాలి. `nums` మొదటి `k` positions లో మిగిలిన elements ఉండాలి (order matter అవ్వదు). `k` దాటిన positions లో ఏమున్నా పట్టించుకోరు.
- **Key constraints:** Extra array వద్దు ($O(1)$ space). Order preserve అవసరం లేదు.
- **ఉదాహరణ:**

Input: `nums = [3,2,2,3]`, `val = 3`
Output: `k = 2`, `nums = [2,2,_,_] (మొదటి 2 elements ముఖ్యం)`

- **ఎలా ఆలోచించాలి:**

"తీసేయడం" (remove) అంటే array లో మధ్యలో నుండి element తీసి, తర్వాతి అన్నిటిని ఎడమకి జరపడం అనుకుంటే – ప్రతి removal $O(n)$, మొత్తం $O(n^2)$. అనవసరం.

ఏం గమనించాలి? మనం నిజంగా "delete" చెయ్యక్కర్లేదు – కేవలం **ఉంచాల్సినవి array మొదటికి వరుసగా జరిపితే** చాలు. అంటే ఇది classic **read/write pointer** (in-place manipulation) పని. `read` pointer అన్నిటిని చూస్తుంది; element `val` కాకపోతే మాత్రమే `write` position కి copy చేసి `write++`. `val` అయితే skip. చివర్లో `write` = ఉంచిన elements count = మన answer `k`.

- **Brute Force:** కొత్త array లో `val` కాని elements push చేసి, తిరిగి copy. Time $O(n)$, కానీ Space $O(n)$ – in-place constraint violate.
- **Optimal Approach: Two pointers (read/write), in-place.** ఒక్క pass, extra space లేదు. Order కూడా preserve అవుతుంది (bonus).
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @param {number} val
 * @return {number} - మిగిలిన elements count (k)
 */
var removeElement = function (nums, val) {
    let write = 0; // తర్వాతి "ఉంచదగ్గ" element ఎక్కడ రాయాలో

    for (let read = 0; read < nums.length; read++) {
        if (nums[read] !== val) { // ఉంచదగ్గ element అయితేనే
            nums[write] = nums[read]; // మొదటికి జరుపు
            write++;
        }
        // nums[read] === val అయితే ఏమీ చెయ్యం - skip (అదే "remove")
    }
}
```

```
return write; // ఇదే k
};
```

- **Dry Run:** `nums=[3,2,2,3]`, `val=3`.

read	nums[read]	val?	action	write	nums
0	3	yes	skip	0	[3,2,2,3]
1	2	no	write nums[0]=2	1	[2,2,2,3]
2	2	no	write nums[1]=2	2	[2,2,2,3]
3	3	yes	skip	2	[2,2,2,3]

Return `k=2`. మొదటి 2 elements `[2,2]` – correct. ✅ (index 2,3 లో ఏమున్నా పట్టించుకోరు.)

- **Complexity:** Time `O(n)` – ఒక్క pass. Space `O(1)`.
- **గుర్తుంచుకోవాల్సినది:** "Remove/filter in-place" = **read/write two-pointer**. `write` pointer answer count ని కూడా ఇస్తుంది. ఈ template Problem 3, 4 లో మళ్ళీ మళ్ళీ కనిపిస్తుంది – ఒక్కసారి పట్టుకో.
 - **Bonus trick:** `val` అరుదుగా వస్తే (చాలా తక్కువ removals), "swap with last" variant వాడొచ్చు – కానీ order break అవుతుంది. Standard read/write safe.
- **సాధారణ తప్పులు:**
 - `write` బదులు `read` ని return చెయ్యడం.
 - `k` దాటిన positions ని "clear" చెయ్యాలని అనవసరంగా ప్రయత్నించడం – problem అడగలేదు.
 - `nums.splice()` వాడడం – అది `O(n)` per call, in-place spirit కి విరుద్ధం, index shift bugs వస్తాయి.

3. Remove Duplicates from Sorted Array (LeetCode #26) – Easy

- **సమస్య:** ఒక **sorted** array `nums` ఇస్తారు. ప్రతి unique element **ఒక్కసారే** ఉండేలా duplicates ని **in-place** గా తీసేసి, unique elements count `k` ని return చెయ్యాలి. మొదటి `k` positions లో unique elements **అదే sorted order** లో ఉండాలి.
- **Key constraints:** Array already sorted (ఇదే మన ఆయుధం). `O(1)` extra space.
- **ఉదాహరణ:**

```
Input:  nums = [0,0,1,1,1,2,2,3,3,4]
```

Output: k = 5, nums = [0,1,2,3,4,...]

- ఎలా ఆలోచించాలి:

Array **sorted** అని పదే పదే చెప్పారు – అంటే duplicates ఎప్పుడూ **పక్కపక్కనే** ఉంటాయి. ఇదే key. Duplicate ని పట్టుకోవడానికి Set/hashing అవసరం లేదు; కేవలం **వెంటనే ముందున్న element తో** పోల్చితే చాలు.

మళ్ళీ read/write pointer. **write** position ఎప్పుడూ "ఇప్పటిదాకా రాసిన చివరి unique element" ని చూపిస్తుంది. **read** కొత్త element **nums[read]** ని, ఇప్పటిదాకా రాసిన చివరి **nums[write-1]** తో పోల్చు. **వేరుగా ఉంటే** అది కొత్త unique → రాయు. మొదటి element ఎప్పుడూ unique కాబట్టి **write** ని **1** నుండి మొదలుపెడతాం.

- **Brute Force:** Set లో వేసి, తిరిగి array కి రాయడం. Time $O(n)$, Space $O(n)$ – sorted property వాడకపోవడం + extra space. సరిపోదు.
- **Optimal Approach: Two pointers, compare with `nums[write-1]`** (in-place). Sorted కాబట్టి neighbor comparison సరిపోతుంది. $O(1)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums - sorted (ascending)
 * @return {number} - unique elements count (k)
 */
var removeDuplicates = function (nums) {
    if (nums.length === 0) return 0; // ఖాళీ array edge case

    let write = 1; // index 0 ఎప్పుడూ unique - దాన్ని ఉంచేసాం, తర్వాత నుండి రాస్తాం

    for (let read = 1; read < nums.length; read++) {
        // ఇప్పటిదాకా రాసిన చివరి unique తో వేరుగా ఉందా?
        if (nums[read] !== nums[write - 1]) {
            nums[write] = nums[read]; // కొత్త unique - జరుపు
            write++;
        }
        // సమానం అయితే duplicate - skip
    }

    return write;
};
```

- **Dry Run:** `nums=[0,0,1,1,1,2,2,3,3,4]` . `write=1` (`nums[0]=0` already unique).

read	nums[read]	nums[write-1]	కొత్తదా?	write	nums (front)
1	0	0	no	1	[0,...]
2	1	0	yes	2	[0,1,...]
3	1	1	no	2	[0,1,...]

read	nums[read]	nums[write-1]	కొత్తదా?	write	nums (front)
4	1	1	no	2	[0,1,...]
5	2	1	yes	3	[0,1,2,...]
6	2	2	no	3	[0,1,2,...]
7	3	2	yes	4	[0,1,2,3,...]
8	3	3	no	4	[0,1,2,3,...]
9	4	3	yes	5	[0,1,2,3,4,...]

Return `k=5` , front = `[0,1,2,3,4]` . ✓

- **Complexity:** Time `O(n)` , Space `O(1)` .
- గుర్తుంచుకోవాల్సినది: "Sorted" = duplicates పక్కపక్కనే = neighbor comparison చాలు, hashing అవసరం లేదు. `nums[write-1]` అనేది "ఇప్పటిదాకా accept చేసిన చివరిది" – ఈ idiom ని పట్టుకో, తర్వాతి problem (at most 2) దీని పైనే build అవుతుంది.
- సాధారణ తప్పులు:
 - `write` ని `0` నుండి మొదలుపెట్టి, `nums[read]` ని `nums[read-1]` తో పోల్చడం (input మారిపోయాక ఇది తప్పు అవుతుంది). ఎప్పుడూ **write-1** (accepted చివరిది) తో పోల్చు, read-1 (original neighbor) తో కాదు.
 - ఖాళీ array `[]` కి `write=1` return చెయ్యడం – guard పెట్టాలి.

4. Remove Duplicates from Sorted Array II (LeetCode #80) – Medium

- **సమస్య:** ఒక **sorted** array `nums` ఇస్తారు. ప్రతి unique element **గరిష్టంగా రెండుసార్లు** ఉండేలా duplicates ని in-place గా తగ్గించి, resulting length `k` ని return చెయ్యాలి. మొదటి `k` positions లో ఆ elements sorted order లో ఉండాలి.
- **Key constraints:** Sorted. `O(1)` space. "At most 2" – ఇదే మార్పు (Problem 3 లో "at most 1" అయ్యింది).
- **ఉదాహరణ:**

Input: `nums = [0,0,1,1,1,1,2,3,3]`

Output: `k = 7`, `nums = [0,0,1,1,2,3,3,...]` (1 నాలుగుసార్లు ఉంది → రెండుకి తగ్గింది)

- ఎలా ఆలోచించాలి:

Problem 3 లో "కొత్త element ని accepted చివరిదాంతో పోల్చాలి." ఇక్కడ "రెండు allowed" – అంటే **accepted చివరి-రెండే element** (`nums[write-2]`) తో పోల్చాలి. ఎందుకు?

అలోచించు: మనం ఒక element రాయాలంటే, అది ఇప్పటిదాకా accept చేసిన **చివరి రెండింటిలో** (`nums[write-1]` , `nums[write-2]`) ఎక్కడైనా మూడోసారి కాకూడదు. Array sorted కాబట్టి, `nums[read] == nums[write-2]` అయితే – write-2, write-1, ఇప్పుడు read – అన్నీ ఒకటే value = మూడోది. కాబట్టి skip. `nums[read] != nums[write-2]` అయితే safe గా రాయొచ్చు.

Generalization (SSE bonus): "at most `t` copies allowed" → `nums[read] != nums[write-t]` అయితేనే రాయి. ఇది beautiful – ఒక్క parameter మార్చితే ఏ limit కైనా పని చేస్తుంది.

- **Brute Force:** ప్రతి value count పెట్టుకుని, $\text{count} \leq 2$ అయ్యేదాకా మాత్రమే copy. HashMap వాడితే Space $O(n)$. సరిపోదు (sorted property వృథా).
- **Optimal Approach:** `nums[read] != nums[write - 2]` check (generalized in-place two-pointer). మొదటి 2 elements ఎప్పుడూ safe (2 కంటే ఎక్కువ ఉండలేవు), కాబట్టి `write=2` నుండి compare మొదలు.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums - sorted
 * @return {number} - resulting length (k)
 */
var removeDuplicates = function (nums) {
    const LIMIT = 2; // ప్రతి value గరిష్ఠంగా ఎన్నిసార్లు
    let write = 0;

    for (let read = 0; read < nums.length; read++) {
        // write < LIMIT: మొదటి 2 slots ఎప్పుడూ safe గా నింపవచ్చు.
        // లేదా: ఇప్పుడు రాయబోయేది, accepted చివరి-2వ దానికంటే వేరుగా ఉంటే (= 3వసారి కాదు).
        if (write < LIMIT || nums[read] !== nums[write - LIMIT]) {
            nums[write] = nums[read];
            write++;
        }
    }

    return write;
};
```

- **Dry Run:** `nums=[0,0,1,1,1,1,2,3,3]` , `LIMIT=2`.

read	nums[read]	write<2?	nums[write-2]	రాయాలా?	write	nums (front)
0	0	yes	–	yes	1	[0]
1	0	yes	–	yes	2	[0,0]
2	1	no	nums[0]=0	1≠0 yes	3	[0,0,1]
3	1	no	nums[1]=0	1≠0 yes	4	[0,0,1,1]

read	nums[read]	write < 2?	nums[write-2]	రాయాలా?	write	nums (front)
4	1	no	nums[2]=1	1==1 no	4	[0,0,1,1]
5	1	no	nums[2]=1	1==1 no	4	[0,0,1,1]
6	2	no	nums[2]=1	2≠1 yes	5	[0,0,1,1,2]
7	3	no	nums[3]=1	3≠1 yes	6	[0,0,1,1,2,3]
8	3	no	nums[4]=2	3≠2 yes	7	[0,0,1,1,2,3,3]

Return `k=7` , front = `[0,0,1,1,2,3,3]` .  (1 నాలుగుసార్లు → రెండుకీ తగ్గింది.)

- **Complexity:** Time `O(n)` , Space `O(1)` .
- **గుర్తుంచుకోవాల్సినది: "At most K copies" pattern** → `nums[read] !== nums[write - K]` . ఇది Problem 3 (K=1) కి కూడా వర్తిస్తుంది – ఒక generalized idea. `write < K` guard మొదటి slots కి. ఈ ఒక్క line మార్పుతో K=1,2,3... ఏదైనా.
- **సాధారణ తప్పులు:**
 - `nums[read-1]` , `nums[read-2]` (original array) తో పోల్చడం – write తో array మారిపోతుంది కాబట్టి **accepted** array (`write` indices) తో పోల్చాలి.
 - `write < LIMIT` guard మర్చిపోతే `nums[write-2] = nums[-1] = undefined` , comparison తప్పు.
 - Nested count loop వాడి `O(n²)` చేయడం – అనవసరం.

5. Majority Element (LeetCode #169) – Easy

- **సమస్య:** ఒక array `nums` (size `n`) ఇస్తారు. అందులో **majority element** ఉంది – అంటే `[n/2]` కంటే ఎక్కువసార్లు (అర్థభాగం దాటి) వచ్చే element. దాన్ని కనుక్కోవాలి. (Majority ఎప్పుడూ ఉంటుందని guarantee.)
- **Key constraints:** Follow-up: `O(n)` time & `O(1)` space లో చెయ్యగలవా?
- **ఉదాహరణ:**

Input: `nums = [2,2,1,1,1,2,2]`
Output: 2 (7 లో 4 సార్లు, 4 > 3)

- **ఎలా ఆలోచించాలి:**

Naive: ప్రతి element ఎన్నిసార్లు వచ్చిందో count చెయ్యి → `O(n²)`. HashMap తో counts → `O(n)` time కానీ `O(n)` space.

Sorting idea: sorted array లో majority element (అర్థభాగం దాటి ఉంది కాబట్టి) **మధ్య index ($n/2$)** దగ్గర కచ్చితంగా ఉంటుంది! `nums.sort(); return nums[Math.floor(n/2)]` . చక్కటి trick, కానీ $O(n \log n)$.

ఏం గమనించాలి? (Boyer-Moore Voting) Majority element మిగతా అందరినీ కలిపినా ఎక్కువ ఉంది. ఒక "వోటింగ్" ఊహించు: majority కి ఒక vote +1, మిగతా ఏ element కైనా -1. మనం ఒక **candidate** పెట్టుకుంటాం, **count** maintain చేస్తాం. **count** 0 అయితే, ప్రస్తుత element ని కొత్త candidate చేసి **count**=1. తర్వాతి element candidate అయితే +1, కాకపోతే -1.

Intuition: majority కాని elements ఒకరినొకరు cancel చేసుకుంటాయి. Majority element మిగతా అందరినీ cancel చేసినా, extra votes మిగిలి **చివరికి అదే candidate గా నిలబడుతుంది**. అర్థభాగం దాటి ఉంది కాబట్టి ఇది కచ్చితంగా పని చేస్తుంది.

- **Brute Force:** HashMap లో frequency count, $> n/2$ దాన్ని return. Time $O(n)$, Space $O(n)$. పని చేస్తుంది కానీ $O(1)$ space కాదు.
- **Optimal Approach: Boyer-Moore Voting Algorithm** (ఒక రకమైన greedy running-state). $O(n)$ time, $O(1)$ space – follow-up కి perfect answer.

• Solution (JavaScript):

```
/**
 * @param {number[]} nums
 * @return {number} - majority element
 */
var majorityElement = function (nums) {
    let candidate = null;
    let count = 0;

    for (const num of nums) {
        if (count === 0) {
            candidate = num; // count సున్నా → కొత్త candidate ని నమ్ముదాం
        }
        // ప్రస్తుత candidate కి vote పడిందా (+1) లేదా వ్యతిరేకమా (-1)?
        count += (num === candidate) ? 1 : -1;
    }

    return candidate; // majority guaranteed కాబట్టి ఇదే answer
};
```

- **Dry Run:** `nums=[2,2,1,1,1,2,2]` .

num	count===0?	candidate	count (after)
2	yes	2	1
2	no	2	2
1	no	2	1
1	no	2	0

num	count==0?	candidate	count (after)
1	yes	1	1
2	no	1	0
2	yes	2	1

చివరి `candidate = 2`. ✓ (2 ఏడు లో నాలుగుసార్లు.)

- **Complexity:** Time `O(n)` – ఒక్క pass. Space `O(1)` – కేవలం రెండు variables.
- **గుర్తుంచుకోవాల్సినది: Boyer-Moore Voting** – "అర్థభాగం దాటి ఉన్న element ని `O(1)` space లో కనుక్కోవడం." "Majority", "more than $n/2$ ", "cancel out" అనే మాటలు వినగానే ఇది గుర్తుకు రావాలి. మధ్య-index sorting trick కూడా జ్ఞాపకం ఉంచు – quick backup answer.
- **సాధారణ తప్పులు:**
 - Majority **guaranteed కాకపోతే** Boyer-Moore ఇచ్చిన candidate నిజంగా majority కాకపోవచ్చు – అప్పుడు రెండో pass లో దాని count verify చెయ్యాలి. ఈ problem లో guaranteed కాబట్టి అవసరం లేదు.
 - `count == 0` అయినప్పుడు candidate update చెయ్యడం మర్చిపోవడం.
 - "Majority = అత్యధికంగా వచ్చేది (mode)" అని పొరపడడం – ఇక్కడ strictly `> n/2` (అర్థభాగం దాటి). Mode majority కాకపోవచ్చు.

6. Rotate Array (LeetCode #189) – Medium

- **సమస్య:** ఒక array `nums` ని కుడివైపుకి `k` steps rotate చెయ్యాలి (`k ≥ 0`). In-place గా చెయ్యగలగాలి.
- **Key constraints:** `k` array size కంటే పెద్దది కావచ్చు (mod తీసుకోవాలి). Follow-up: `O(1)` extra space.
- **ఉదాహరణ:**

Input: `nums = [1,2,3,4,5,6,7]`, `k = 3`
Output: `[5,6,7,1,2,3,4]` (చివరి 3 elements ముందుకి వచ్చాయి)

- **ఎలా ఆలోచించాలి:**

మొదటి ఆలోచన: ఒక్కో step చొప్పున `k` సార్లు rotate – ప్రతిసారీ చివరి element ముందుకి తేవడం `O(n)`, మొత్తం `O(n·k)`. Slow.

రెండో ఆలోచన: కొత్త array లో `newArr[(i + k) % n] = nums[i]`. `O(n)` time కానీ `O(n)` space.

ఏం గమనించాలి? (Reversal trick) కుడికి `k` rotate అంటే – చివరి `k` elements ముందుకి, మిగతా `n-k` elements వెనక్కి దీన్ని reverse లతో చేయవచ్చు, ఇదొక classic:

1. మొత్తం array ని reverse చెయ్యి → చివరివి ముందుకి వస్తాయి కానీ తప్పు order లో.
2. మొదటి k ని reverse చెయ్యి → వాటి order సరిచేస్తుంది.
3. మిగతా $n-k$ ని reverse చెయ్యి → వాటి order సరిచేస్తుంది.

ఎందుకు పని చేస్తుంది? Reverse-all తో రెండు blocks position లు swap అవుతాయి (కుడి block ఎడమకి వచ్చేస్తుంది), కానీ ప్రతి block లోపల order తలకిందులు. మళ్ళీ ఒక్కో block reverse చేస్తే లోపల order సరైపోతుంది. **మొదట $k \% = n$** చెయ్యడం మర్చిపోవద్దు – k పెద్దదైతే.

- **Brute Force:** ఒక్కో step rotate, k సార్లు. Time $O(n \cdot k)$, Space $O(1)$. TLE అవుతుంది పెద్ద inputs కి.
- **Optimal Approach: Reverse trick (3 reversals)** – in-place, $O(n)$ time, $O(1)$ space. (Extra-array version $O(n)$ space – follow-up కోసం reversal ప్రాధాన్యం.)
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {void} - in-place
 */
var rotate = function (nums, k) {
    const n = nums.length;
    k %= n; // k > n అయితే wrap; k=0 అయితే ఏమీ మారదు
    if (k === 0) return;

    reverse(nums, 0, n - 1); // 1) మొత్తం reverse
    reverse(nums, 0, k - 1); // 2) మొదటి k reverse
    reverse(nums, k, n - 1); // 3) మిగతా reverse
};

// [l..r] range ని in-place నా reverse చేసే helper (two pointers)
function reverse(nums, l, r) {
    while (l < r) {
        [nums[l], nums[r]] = [nums[r], nums[l]]; // destructuring swap
        l++;
        r--;
    }
}
```

- **Dry Run:** $nums=[1,2,3,4,5,6,7]$, $k=3$. $n=7$, $k=3$.

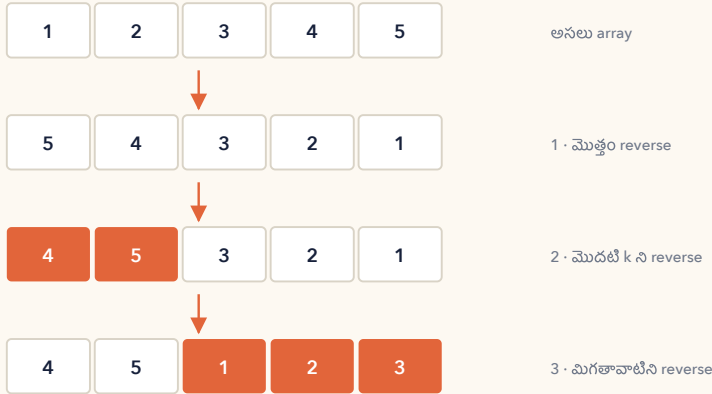
step	operation	result
start	–	[1,2,3,4,5,6,7]
1	reverse(0,6)	[7,6,5,4,3,2,1]
2	reverse(0,2)	[5,6,7,4,3,2,1]
3	reverse(3,6)	[5,6,7,1,2,3,4]

Result [5,6,7,1,2,3,4] . ✓

- **Complexity:** Time $O(n)$ – మూడు reversals, ప్రతిదీ linear. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది: Array rotation = 3 reversals.** "Reverse whole → reverse parts" idiom ని గట్టిగా పట్టుకో – string rotation, "rotate words" లాంటి problems కి కూడా వస్తుంది. ఎప్పుడూ $k \% = n$ మొదట.
- **సాధారణ తప్పులు:**
 - $k \% = n$ మర్చిపోవడం → $k > n$ అయితే index out of range లేదా తప్పు answer.
 - $n = 0$ అయితే $k \% = 0 = \text{NaN}$ – ఖాళీ array guard అవసరం (constraints సాధారణంగా $n \geq 1$).
 - కుడి vs ఎడమ rotation గందరగోళం – కుడికి k rotate = ఎడమకి $(n-k)$ rotate.

ROTATE ARRAY · మూడు REVERSALS TRICK

ROTATE ARRAY $k=2$ · మూడు reversals



ఎందుకు ఇది పని చేస్తుంది

Reverse చేయడం వల్ల చివరి k elements ముందుకి వస్తాయి – కానీ తలక్రిందులుగా.
ఆ రెండు ముక్కలని ఏడిగా మళ్ళీ reverse చేస్తే అవి సరైన క్రమానికి వస్తాయి.
 $O(1)$ space · $O(n)$ time – కేత్ర array అవసరం లేదు.

$k > n$ అయితే – $k \% = n$ చేయడం మర్చిపోవద్దు, లేకపోతే index దాటిపోతుంది. ఇది interview lo అత్యంత సాధారణంగా మిస్ అయ్యే line.

7. Best Time to Buy and Sell Stock (LeetCode #121) – Easy

- **సమస్య:** $\text{prices}[i] = i$ వ రోజు stock ధర. ఒక్కసారి కొని (ఒక రోజు), తర్వాత ఒక రోజు అమ్మి – గరిష్ట profit ఎంత? అమ్మేది కొన్న తర్వాత ఉండాలి. Profit రాకపోతే 0 return.
- **Key constraints:** ఒకే transaction (ఒక buy + ఒక sell). Sell day > buy day.
- **ఉదాహరణ:**

Input: $\text{prices} = [7, 1, 5, 3, 6, 4]$
Output: 5 (day 2 న 1 కి కొని, day 5 న 6 కి అమ్మితే $6 - 1 = 5$)

- ఎలా ఆలోచించాలి:

Naive: ప్రతి జత (i, j), $i < j$ కి $prices[j] - prices[i]$ గరిష్ఠం $\rightarrow O(n^2)$.

ఏం గమనించాలి? ఏ రోజైనా అమ్మాలంటే, గరిష్ఠ profit కోసం **అప్పటిదాకా వచ్చిన కనిష్ఠ ధరకి** కొని ఉండాలి. అంటే array మీద ఒక్కసారి నడుస్తూ, "ఇప్పటిదాకా చూసిన కనిష్ఠ ధర" (`minPrice`) ని track చెయ్యి. ప్రతి రోజు, "ఈ రోజు అమ్మితే $profit = today - minPrice$ " లెక్కించి, గరిష్ఠం update చెయ్యి.

కీలక realization: buy ఎప్పుడూ **గతంలో** జరిగింది (minPrice అనేది ఇప్పటిదాకా చూసినది మాత్రమే), కాబట్టి "sell after buy" constraint దానంతట అదే satisfy అవుతుంది. ఒక్క pass సరిపోతుంది.

- **Brute Force:** అన్ని (buy, sell) జతలు. Time $O(n^2)$, Space $O(1)$. TLE పెద్ద inputs కి.
- **Optimal Approach: Single-pass greedy – running minimum + running max profit.** ప్రతి రోజు "ఇప్పటిదాకా కనిష్ఠ ధర" గుర్తుంచుకో, ఈ రోజు అమ్మే profit తో max update. $O(n)$ time, $O(1)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} prices
 * @return {number} - గరిష్ఠ profit (లేకపోతే 0)
 */
var maxProfit = function (prices) {
    let minPrice = Infinity; // ఇప్పటిదాకా చూసిన కనిష్ఠ కొనుగోలు ధర
    let maxProfit = 0; // ఇప్పటిదాకా సాధ్యమైన గరిష్ఠ profit

    for (const price of prices) {
        if (price < minPrice) {
            minPrice = price; // ఇంకా చౌక ధర దొరికింది → ఇక్కడ కొంటే మంచిది
        } else if (price - minPrice > maxProfit) {
            maxProfit = price - minPrice; // ఈ రోజు అమ్మితే better profit
        }
    }

    return maxProfit;
};
```

- **Dry Run:** `prices=[7,1,5,3,6,4]` .

price	minPrice (after)	price-minPrice	maxProfit (after)
7	7	–	0
1	1	–	0
5	1	4	4
3	1	2	4
6	1	5	5
4	1	3	5

Return 5 . 

- **Complexity:** Time $O(n)$, Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది: "గతంలో min, ఇప్పుడు sell" – running-min greedy.** "గరిష్ట difference where smaller comes first" అనే ఏ problem కైనా ఈ pattern. minPrice భవిష్యత్తు చూడదు, గతమే చూస్తుంది – అందుకే constraint automatic గా satisfy.
- **సాధారణ తప్పులు:**
 - `Math.max(...prices) - Math.min(...prices)` – తప్పు! Min, Max order తప్పు కావచ్చు (min max తర్వాత వస్తే).
 - `maxProfit` ని `-Infinity` తో మొదలుపెట్టడం – profit లేకపోతే `0` return చెయ్యాలి, negative కాదు.
 - `else if` బదులు రెండు separate `if` వాడినా పని చేస్తుంది, కానీ same day లో min update + sell రెండూ అనవసరం (profit 0). Logic గా రెండూ correct.

8. Best Time to Buy and Sell Stock II (LeetCode #122) – Medium

- **సమస్య:** `prices[i]` = `i` వ రోజు ధర. ఇప్పుడు ఎన్నిసార్లైనా కొని-అమ్మవచ్చు (ఒకే రోజు అమ్మి మళ్ళీ కొనవచ్చు కూడా). కానీ ఒకేసారి గరిష్టంగా ఒక stock మాత్రమే holdలో ఉండాలి (కొన్నది అమ్మాకే మళ్ళీ కొనాలి). గరిష్ట total profit?
- **Key constraints:** Multiple transactions. ఒకేసారి ఒక share.
- **ఉదాహరణ:**

```
Input: prices = [7,1,5,3,6,4]
Output: 7   (1→5 profit 4, 3→6 profit 3, total 7)
```

- **ఎలా ఆలోచించాలి:**

Problem 7 లో ఒకే transaction. ఇక్కడ unlimited. DP తో "hold / not-hold" states తో చెయ్యవచ్చు – కానీ చాలా simple greedy ఉంది.

ఏం గమనించాలి? Total profit ని రోజువారీ price మార్పుల మొత్తంగా విడదీయవచ్చు. Price పెరిగే ప్రతి consecutive రోజు జతని ఒక micro-transaction గా చూడు. ఉదా: `1→5` అంటే `(1→3) + (3→5)` – పెరుగుదల అంతా కలిపితే అదే. కాబట్టి ప్రతి `prices[i] > prices[i-1]` ఉన్నప్పుడల్లా ఆ difference ని profit కి కలిపేయి. అన్ని upward slopes ని పట్టుకుంటే గరిష్ట profit దానంతట అదే.

Analogy: ఒక పర్వత శ్రేణి. లోయలో కొని, శిఖరం దగ్గర అమ్ముతావు. ప్రతి "ఎక్కుడు" segment ని పట్టుకోవడం = ప్రతి positive daily change ని పట్టుకోవడం. Downward slopes ని మనం అస్సలు hold చేయం.

- **Brute Force:** అన్ని transaction combinations recursive గా try (buy/sell/skip ప్రతి రోజు) → exponential, లేదా DP $O(n)$. DP correct కానీ greedy చాలా simple.
- **Optimal Approach: Greedy – sum of all positive consecutive differences.** $O(n)$ time, $O(1)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} prices
 * @return {number} - గరిష్ట total profit
 */
var maxProfit = function (prices) {
    let profit = 0;

    for (let i = 1; i < prices.length; i++) {
        // నిన్నటికంటే ఈ రోజు ధర ఎక్కువ అయితే - ఆ పెరుగుదలని జేబులో వేసుకో
        if (prices[i] > prices[i - 1]) {
            profit += prices[i] - prices[i - 1];
        }
    }

    return profit;
};
```

- **Dry Run:** prices=[7,1,5,3,6,4] .

i	prices[i-1]	prices[i]	పెరిగిందా?	add	profit
1	7	1	no	0	0
2	1	5	yes	4	4
3	5	3	no	0	4
4	3	6	yes	3	7
5	6	4	no	0	7

Return 7.  (1→5 = 4, 3→6 = 3.)

- **Complexity:** Time $O(n)$, Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** "Unlimited transactions → అన్ని positive daily deltas కలిపేయి." పెద్ద slope ని చిన్న slopes గా విడదీసినా total అదే - ఈ decomposition insight చాలా greedy problems కి ఉపయోగం. Problem 7 (single) vs 8 (unlimited) తేడా గుర్తుంచుకో: constraint మారితే approach పూర్తిగా మారుతుంది.
- **సాధారణ తప్పులు:**
 - Global min కి కొని global max కి అమ్మాలనుకోవడం - multiple transactions లో అది suboptimal.

- Transaction fee/cooldown ఉన్న variants లో ఈ simple greedy పని చెయ్యదు – అవి DP అవసరం. (Plain #122 కి greedy perfect.)
- `i=0` నుండి loop మొదలుపెట్టి `prices[-1]` access చెయ్యడం – `i=1` నుండి మొదలుపెట్టాలి.

9. Jump Game (LeetCode #55) – Medium

- సమస్య:** Array `nums`, ప్రతి `nums[i]` = ఆ index నుండి గరిష్ఠంగా ఎంత దూరం ముందుకి jump చెయ్యగలవో. Index `0` నుండి మొదలై, చివరి index చేరగలవా? `true / false`.
- Key constraints:** ప్రతి jump 0 నుండి `nums[i]` వరకు ఏదైనా కావచ్చు. `nums[i]=0` అంటే అక్కడ చిక్కుకుపోతావు.
- ఉదాహరణ:**

Input: `nums = [2,3,1,1,4]` → `true` (`0→1→4`, లేదా `0→2→...`)
 Input: `nums = [3,2,1,0,4]` → `false` (index 3 దగ్గర 0, దాటలేం)

- ఎలా ఆలోచించాలి:**

Naive: ప్రతి index నుండి సాధ్యమైన అన్ని jumps recursive గా try (backtracking/DP) → exponential లేదా $O(n^2)$.

ఏం గమనించాలి? మనం ఏ path తీసుకున్నామో అనవసరం – కేవలం "ఇప్పటిదాకా నేను ఎంత దూరం చేరగలను" (`farthest`) అనేది ముఖ్యం. Array మీద ఎడమ నుండి కుడికి నడుస్తూ, ప్రతి index `i` దగ్గర: మొదట "నేను అసలు ఇక్కడికి చేరగలనా?" (`i <= farthest`) check చెయ్యి. చేరగలిగితే, `farthest = max(farthest, i + nums[i])` update చెయ్యి (ఇక్కడ నుండి ఇంకా దూరం వెళ్ళగలనేమో). ఏ index దగ్గరైనా `i > farthest` అయితే – అక్కడికి చేరనేలేను → `false`. Loop పూర్తయితే చివరిదాకా చేరగలం → `true`.

Analogy: పెట్రోల్ ఉన్న కారు. `farthest` = ఇప్పటి ఇంధనంతో వెళ్ళగల గరిష్ఠ దూరం. దారిలో ప్రతి petrol bunk (index) ఇంకా ఇంధనం ఇస్తుంది (`i + nums[i]`). ఏ point దగ్గరైనా reach దాటిపోతే – ఆగిపోతావు.

- Brute Force:** DP `canReach[i]`, ప్రతి `i` కి అన్ని previous jumps check → $O(n^2)$, TLE.
- Optimal Approach: Greedy – track farthest reachable index in one pass.** $O(n)$ time, $O(1)$ space.
- Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @return {boolean}
 */
var canJump = function (nums) {
  let farthest = 0; // ఇప్పటిదాకా చేరగల గరిష్ఠ index
```

```

for (let i = 0; i < nums.length; i++) {
    if (i > farthest) return false; // ఈ index కి అస్సలు చేరలేను
    farthest = Math.max(farthest, i + nums[i]); // ఇక్కడ నుండి ఇంకా దూరం?
    if (farthest >= nums.length - 1) return true; // చివరిది చేరేసా (early exit)
}

return true; // అన్ని indices దాటాం → చేరగలం
};

```

- **Dry Run:** `nums=[2,3,1,1,4]` , last index = 4.

i	i>farthest?	i+nums[i]	farthest (after)	≥4?
0	0>0 no	0+2=2	2	no
1	1>2 no	1+3=4	4	yes → true

Return `true`. ✅ రెండో ఉదాహరణ `[3,2,1,0,4]` : index 0→farthest=3, i=1→farthest=max(3,3)=3, i=2→max(3,3)=3, i=3→max(3,3)=3, i=4→ `4 > 3` → `false`. ✅

- **Complexity:** Time `O(n)` , Space `O(1)` .
- **గుర్తుంచుకోవాల్సినది: "Reachability = running farthest reach."** Path ని track చెయ్యకు, **reach** ని track చెయ్యి. `i > farthest` = "ఇక్కడ ఇరుక్కుపోయా" signal. ఈ farthest-reach idea Problem 10 (min jumps) కి foundation.
- **సాధారణ తప్పులు:**
 - Backtracking/DP వాడి `O(2^n)/O(n^2)` – greedy `O(n)` చాలు.
 - `i > farthest` check ని `farthest` update తర్వాత పెట్టడం – ముందు check చెయ్యాలి (ఈ index reachable అని నిర్ధారించాకే దాన్ని ఉపయోగించాలి).
 - `nums[i]=0` ఉన్న చోట logic మర్చిపోవడం – farthest update అవదు, తర్వాత stuck అవుతుందేమో check అవుతుంది.

10. Jump Game II (LeetCode #45) – Medium

- **సమస్య:** అదే Jump Game – కానీ ఇప్పుడు చివరి index కి చేరడానికి **కనిష్ట jumps** ఎన్ని కావాలో కనుక్కోవాలి. చేరడం guaranteed (test cases).
- **Key constraints:** ఎప్పుడూ చేరగలం. కనిష్ట jump count కావాలి.
- **ఉదాహరణ:**

```

Input:  nums = [2,3,1,1,4]
Output: 2    (index 0→1 (jump 1), 1→4 (jump 2))

```

• ఎలా ఆలోచించాలి:

BFS లా ఆలోచించు: "0 jumps తో నేను index 0 లో ఉన్నా. 1 jump తో ఎక్కడెక్కడ చేరగలను? 2 jumps తో ఎక్కడెక్కడ?" ప్రతి "level" ఒక jump. చివరి index ఏ level లో వస్తుందో అదే answer. కానీ explicit BFS queue అవసరం లేదు – greedy గా levels ని windows లా చూడవచ్చు.

ఏం గమనించాలి? (Greedy BFS / implicit levels) ప్రస్తుత jump తో చేరగల range `[start...curEnd]` .

ఆ range అంతా నడుస్తూ, "వచ్చే jump లో ఎంత దూరం చేరగలను" (`farthest`) లెక్కించు. ప్రస్తుత range చివర (`i === curEnd`) కి వచ్చినప్పుడు – ఒక jump commit చెయ్యాలి (`jumps++`), కొత్త range చివర `curEnd = farthest` . అంటే: ఒక window లో ఉన్నంతసేపు అదే jump; window అయిపోతే jump పెరుగుతుంది.

కీలకం: loop ని `n-1` వరకే తిప్పు (చివరి index చేరాక కొత్త jump అవసరం లేదు, లేకపోతే overcount).

- **Brute Force:** DP `minJumps[i]` = i కి చేరే కనిష్ట jumps, ప్రతి i కి అన్ని reachable ముందు indices నుండి $\rightarrow O(n^2)$, పెద్ద inputs కి slow.
- **Optimal Approach: Greedy BFS-by-levels – curEnd (ఈ jump range) + farthest (వచ్చే jump reach).** $O(n)$ time, $O(1)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @return {number} - కనిష్ట jumps
 */
var jump = function (nums) {
    let jumps = 0;    // ఇప్పటిదాకా చేసిన jumps
    let curEnd = 0;   // ప్రస్తుత jump తో చేరగల చివరి index (ఈ "level" సరిహద్దు)
    let farthest = 0; // ఈ level లో ఎక్కడి నుండైనా చేరగల గరిష్ట index

    // n-1 వరకే: చివరి index చేరాక ఇంకొక jump అవసరం లేదు
    for (let i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);

        if (i === curEnd) {    // ప్రస్తుత level చివరికి వచ్చాం → jump చెయ్యాలి
            jumps++;
            curEnd = farthest; // కొత్త level సరిహద్దు
        }
    }

    return jumps;
};
```

- **Dry Run:** `nums=[2,3,1,1,4]` , loop `i=0...3` .

i	i+nums[i]	farthest	i===curEnd?	jumps	curEnd (after)
0	2	2	0===0 yes	1	2
1	4	4	1===2 no	1	2

i	i+nums[i]	farthest	i==curEnd?	jumps	curEnd (after)
2	3	4	2==2 yes	2	4
3	4	4	3==4 no	2	4

Return 2. 

- **Complexity:** Time $O(n)$, Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** "కనిష్ట steps in a graph/array = BFS by levels." ఇక్కడ level = ఒక jump. `curEnd` (current level boundary) + `farthest` (next level reach) – ఈ two-variable greedy BFS pattern చాలా "minimum jumps/steps" problems కి వస్తుంది. `n-1` loop bound off-by-one ని జాగ్రత్తగా గుర్తుంచుకో.
- **సాధారణ తప్పులు:**
 - Loop ని `n` వరకు తిప్పడం → చివరి index దగ్గర ఇంకో jump count అయి, answer 1 ఎక్కువ.
 - `curEnd` update ని `i === curEnd` బయట పెట్టడం – ప్రతి step jump అవుతుంది, తప్పు.
 - `farthest` ని jump commit తర్వాతే update చెయ్యడం – window అంతా scan చేసిన farthest వాడాలి, కాబట్టి update ముందు, decision తర్వాత.

11. H-Index (LeetCode #274) – Medium

- **సమస్య:** `citations[i]` = ఒక researcher యొక్క `i` వ paper కి వచ్చిన citations. **h-index** ని కనుక్కోవాలి: గరిష్ట `h` value అంటే – ఆ researcher కి **కనీసం `h` papers** ఉండాలి, వాటిలో ప్రతిదీ **కనీసం `h` citations** పొందాలి.
- **Key constraints:** Order లేదు. Follow-up: $O(n)$ time.
- **ఉదాహరణ:**

Input: `citations = [3,0,6,1,5]`

Output: 3 (3 papers కి ≥ 3 citations ఉన్నాయి: 3,6,5. 4వది కావాలంటే ≥ 4 citations ఉన్న 4 papers కావాలి – లేవు)

- **ఎలా ఆలోచించాలి:**

Definition ముందు జీర్ణించుకో: h-index `h` అంటే "నా top `h` papers ప్రతి ఒక్కటి కనీసం `h` సార్లు cite అయ్యాయి." పెద్ద `h` మంచిది.

Naive: `h` ని `n` నుండి 0 వరకు try చెయ్యి, ప్రతి `h` కి " $\geq h$ citations ఉన్న papers ఎన్ని?" count చెయ్యి → $O(n^2)$. లేదా descending sort చేసి, sorted array లో `citations[i] >= i+1` ఉన్నంతవరకు `h` పెంచు → $O(n \log n)$, చక్కటి approach.

ఏం గమనించాలి? (Counting sort – $O(n)$) Citations n కంటే ఎక్కువ ఉన్నా, h-index గరిష్టంగా n మాత్రమే (papers n ఉన్నాయి). కాబట్టి citation count ని n దగ్గర **cap** చేసి, ప్రతి count కి ఎన్ని papers ఉన్నాయో buckets లో లెక్కించు. తర్వాత $i = n$ నుండి 0 కి దిగుతూ, " $\geq i$ citations ఉన్న papers" (suffix sum) కూడబెట్టు. ఆ count $\geq i$ అయిన మొదటి i – అదే h-index (అదే గరిష్ట h).

- **Brute Force:** ప్రతి సాధ్యమైన h కి papers count $\rightarrow O(n^2)$. లేదా sort $\rightarrow O(n \log n)$.
- **Optimal Approach: Counting sort (bucket) + suffix accumulation.** citation count ని n దగ్గర cap చెయ్యడమే key insight. $O(n)$ time, $O(n)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} citations
 * @return {number} - h-index
 */
var hIndex = function (citations) {
    const n = citations.length;
    // buckets[c] = సరిగ్గా c citations ఉన్న papers count (c >= n అయితే bucket n లో వేయి)
    const buckets = new Array(n + 1).fill(0);

    for (const c of citations) {
        buckets[Math.min(c, n)]++; // n దాటిన citations అన్న bucket[n] కి
    }

    // i = n నుండి దిగుతూ, ">= i citations ఉన్న papers" మొత్తం కూడబెట్టు
    let count = 0;
    for (let i = n; i >= 0; i--) {
        count += buckets[i];
        if (count >= i) return i; // >= i papers, ప్రతిదీ >= i citations  $\rightarrow$  h = i
    }

    return 0;
};
```

- **Dry Run:** citations=[3,0,6,1,5], n=5.

Buckets build (cap at 5): 3 $\rightarrow$ b[3], 0 $\rightarrow$ b[0], 6 $\rightarrow$ b[5], 1 $\rightarrow$ b[1], 5 $\rightarrow$ b[5]. buckets = [1,1,0,1,0,2] (index 0..5).

i	buckets[i]	count ($\geq i$ papers)	count $\geq i$?
5	2	2	2 $\geq$ 5 no
4	0	2	2 $\geq$ 4 no
3	1	3	3 $\geq$ 3 yes $\rightarrow$ 3

Return 3. 

- **Complexity:** Time $O(n)$ – buckets build $O(n)$ + suffix scan $O(n)$. Space $O(n)$ – buckets array. (Sort approach: $O(n \log n)$ time, $O(1)$ space – interview లో రెండూ discuss చెయ్యి.)

- గుర్తుంచుకోవాల్సినది: "Value range bounded (ఇక్కడ $h \leq n$) → counting sort $O(n)$. Value ని n దగ్గర cap చెయ్యడం (ఎందుకంటే $h > n$ కావు) అనేది key trick. "at least k items with property $\geq k$ " లాంటి threshold problems కి suffix-sum-over-buckets ఒక powerful tool.

- సాధారణ తప్పులు:

- Citations ని cap చెయ్యకపోతే bucket array చాలా పెద్దది (citation విలువలు huge అవ్వచ్చు) – memory waste. `Math.min(c, n)` ముఖ్యం.
- Suffix accumulation direction తప్పు (ascending నుండి) – descending (`i=n-0`) కావాలి, " $\geq i$ " అర్థం కోసం.
- Sort approach లో `citations[i] >= n - i` (descending index math) తికమక – counting sort clearer.

12. Insert Delete GetRandom $O(1)$ (LeetCode #380) – Medium

- సమస్య: ఒక `RandomizedSet` data structure design చెయ్యి, ఈ మూడూ **average $O(1)$** లో:
 - `insert(val)` – లేకపోతే add చేసి `true`, ఉంటే `false`.
 - `remove(val)` – ఉంటే తీసేసి `true`, లేకపోతే `false`.
 - `getRandom()` – ప్రస్తుత elements లో ఒకటి **uniform random** గా return (ప్రతిదానికి సమాన probability).
- **Key constraints:** మూడూ $O(1)$. Random uniform.
- ఉదాహరణ:

```
insert(1)→true, remove(2)→false, insert(2)→true,
getRandom()→1 లేదా 2 (random), remove(1)→true, insert(2)→false, getRandom()→2
```

- ఎలా ఆలోచించాలి:

ఏ ఒక్క structure ఇవన్నీ ఇవ్వదు:

- **HashSet/Map:** insert, remove, lookup $O(1)$. కానీ `getRandom` – set లో "iవ element" $O(1)$ లో దొరకదు (index లేదు). Random కి $O(n)$.
- **Array:** index తో $O(1)$ random దొరుకుతుంది. కానీ `remove(val)` – value వెతికి తీసేయడం $O(n)$, పైగా middle నుండి తీస్తే shift $O(n)$.

ఏం గమనించాలి? (రెండింటినీ కలుపు) Array **fast random** ఇస్తుంది, HashMap **fast lookup** ఇస్తుంది. కలుపు:

- `arr` – values ని hold చేస్తుంది (`getRandom` కి index access).
- `map` – value → arr లో దాని index (lookup, remove కి).

Remove లో $O(1)$ shift ని ఎలా తప్పించాలి? **Swap-with-last trick**: తీసేయాల్సిన element ని array లోని చివరి element తో swap చేసి, `pop()` చెయ్యి (చివరి నుండి తీయడం $O(1)$). Map లో swap అయిన element కొత్త index update చెయ్యి. Middle shift లేకుండా remove $O(1)$!

- **Brute Force**: ఒక్క array – insert/getRandom $O(1)$ కానీ remove/contains $O(n)$. లేదా ఒక్క Set – getRandom $O(n)$. రెండూ fail follow-up.
- **Optimal Approach: Array + HashMap combo, remove లో swap-with-last**. అన్ని ops average $O(1)$.
- **Solution (JavaScript)**:

```
var RandomizedSet = function () {
  this.arr = []; // values (index access కోసం)
  this.map = new Map(); // value -> arr లో index
};

/** @param {number} val @return {boolean} */
RandomizedSet.prototype.insert = function (val) {
  if (this.map.has(val)) return false; // already ఉంది
  this.map.set(val, this.arr.length); // కొత్త element index = ప్రస్తుత length
  this.arr.push(val);
  return true;
};

/** @param {number} val @return {boolean} */
RandomizedSet.prototype.remove = function (val) {
  if (!this.map.has(val)) return false;

  const idx = this.map.get(val); // తీసేయాల్సిన index
  const lastVal = this.arr[this.arr.length - 1];

  // చివరి element ని తీసేయాల్సిన చోటికి తెచ్చి, map update
  this.arr[idx] = lastVal;
  this.map.set(lastVal, idx);

  // ఇప్పుడు చివరిది duplicate – pop చేసి, val ని map నుండి తీసేయి
  this.arr.pop();
  this.map.delete(val);
  return true;
};

/** @return {number} */
RandomizedSet.prototype.getRandom = function () {
  const idx = Math.floor(Math.random() * this.arr.length);
  return this.arr[idx]; // uniform random
};
```

- **Dry Run**: `insert(1)` → `map{1:0}`, `arr[1]`, true. `insert(2)` → `map{1:0,2:1}`, `arr[1,2]`, true.
`remove(1)` :
 - `idx=0`, `lastVal=2`. `arr[0]=2` → `arr[2,2]`. `map.set(2,0)` → `map{1:0,2:0}`.
 - `arr.pop()` → `arr[2]`. `map.delete(1)` → `map{2:0}`. Return true.

- `getRandom()` → `arr[0]=2` (ఒక్కటే element). ✅ Swap-with-last సరిగ్గా middle shift తప్పించింది.
- **Complexity:** insert/remove/getRandom అన్న **average $O(1)$** (HashMap ops amortized $O(1)$, array push/pop $O(1)$). Space **$O(n)$** .
- **గుర్తుంచుకోవాల్సినది: " $O(1)$ random access + $O(1)$ remove = Array (random) + HashMap (value→index), remove లో swap-with-last."** ఈ combo design pattern చాలా "insert/delete/getRandom $O(1)$ " variants కి (duplicates allowed version #381 కూడా) పునాది. Middle నుండి $O(1)$ remove కావాలంటే ఎప్పుడూ **చివరితో swap చేసి pop** ఆలోచించు.
- **సాధారణ తప్పులు:**
 - Swap తర్వాత `map.set(lastVal, idx)` update మర్చిపోవడం → stale index, తర్వాత corruption.
 - తీసేయాల్సినది **already చివరి element** అయితే – swap self తో అవుతుంది, pop తర్వాత `map.delete(val)` సరిగ్గా పని చేస్తుంది (order ముఖ్యం: set తర్వాత delete). ఈ code దీన్ని safe గా handle చేస్తుంది.
 - `arr.splice(idx,1)` వాడడం → $O(n)$ shift, follow-up fail.

13. Product of Array Except Self (LeetCode #238) – Medium

- **సమస్య:** Array `nums` ఇస్తారు. `answer[i] = nums[i]` తప్ప మిగతా అన్ని elements product. **Division వాడకూడదు**, మరియు **$O(n)$** లో చెయ్యాలి.
- **Key constraints:** Division నిషిద్ధం (zero వల్ల కూడా). Output array extra space గా లెక్కించరు (follow-up: $O(1)$ extra).
- **ఉదాహరణ:**

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]` (`answer[0]=2*3*4=24`, `answer[1]=1*3*4=12`, ...)

- **ఎలా ఆలోచించాలి:**

Naive: ప్రతి i కి, మిగతా అన్నిటి product ని inner loop తో → $O(n^2)$.

Division idea: మొత్తం product / `nums[i]`. కానీ **నిషిద్ధం** (పైగా `nums[i]=0` అయితే division fail).

ఏం గమనించాలి? (Prefix × Suffix) `answer[i] = (i కి ఎడమవైపు అన్నిటి product) × (i కి కుడివైపు అన్నిటి product)`. ఈ రెండింటినీ ముందే లెక్కిస్తే ప్రతి i కి $O(1)$.

- మొదటి pass (ఎడమ→కుడి): `answer[i]` లో " i కి ఎడమవైపు product" (prefix) పెట్టు.
- రెండో pass (కుడి→ఎడమ): ఒక running `suffix` maintain చేస్తూ, `answer[i] *= suffix` (i కి కుడివైపు product ని multiply).

ఇలా చేస్తే prefix/suffix arrays వేరుగా అవసరం లేదు – output array + ఒక్క variable సరిపోతుంది. $O(1)$ extra space (output తప్ప).

- **Brute Force:** ప్రతి i కి nested product $\rightarrow O(n^2)$, TLE.
- **Optimal Approach: Prefix/Suffix products pattern, two passes, $O(1)$ extra space.** Output array లోనే prefix build చేసి, తర్వాత running suffix తో multiply.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @return {number[]}
 */
var productExceptSelf = function (nums) {
    const n = nums.length;
    const answer = new Array(n);

    // Pass 1: answer[i] = i కి ఎడమవైపు అన్నిటి product (prefix)
    answer[0] = 1; // index 0 కి ఎడమవైపు ఏమీ లేదు  $\rightarrow 1$  (identity)
    for (let i = 1; i < n; i++) {
        answer[i] = answer[i - 1] * nums[i - 1];
    }

    // Pass 2: కుడి నుండి running suffix ని multiply
    let suffix = 1; // చివరి index కి కుడివైపు ఏమీ లేదు  $\rightarrow 1$ 
    for (let i = n - 1; i >= 0; i--) {
        answer[i] *= suffix; // ఎడమ product  $\times$  కుడి product
        suffix *= nums[i]; // suffix ని update (ఈ element ని కుడి product లోకి)
    }

    return answer;
};
```

- **Dry Run:** `nums=[1,2,3,4]` .

Pass 1 (prefix): `answer[0]=1, answer[1]=1*1=1, answer[2]=1*2=2, answer[3]=2*3=6  $\rightarrow$  [1,1,2,6]` .

Pass 2 (suffix), suffix starts 1:

i	answer[i] (before)	suffix	answer[i]*=suffix	suffix*=nums[i]
3	6	1	6	$1*4=4$
2	2	4	8	$4*3=12$
1	1	12	12	$12*2=24$
0	1	24	24	$24*1=24$

Result `[24,12,8,6]` . 

- **Complexity:** Time $O(n)$ – two linear passes. Space $O(1)$ extra (output array లెక్కించకపోతే).
- **గుర్తుంచుకోవాల్సినది:** "ప్రతి index కి left-contribution $\times$ right-contribution $\rightarrow$ Prefix/Suffix pattern." Division లేకుండా, output array ను prefix కి reuse చేసి, running suffix తో overlay చెయ్యడం – space optimize చేసే classic technique. "except self", "both sides product/sum" అనగానే ఇది.
- **సాధారణ తప్పులు:**
 - `answer[0]` ని `nums[0]` గా పెట్టడం – ఎడమవైపు ఏమీ లేదు కాబట్టి 1 (identity).
 - Division వాడి zero edge case లో crash (nums లో ఒకటి 0 అయితే మొత్తం product 0, division undefined).
 - రెండు separate `prefix[]`, `suffix[]` arrays వాడి $O(n)$ extra space – output + variable తో $O(1)$ చెయ్యొచ్చు.

14. Gas Station (LeetCode #134) – Medium

- **సమస్య:** వృత్తాకార route లో n gas stations. Station i దగ్గర `gas[i]` పెట్రోల్ ఉంది. Station i నుండి $i+1$ కి వెళ్ళడానికి `cost[i]` పెట్రోల్ ఖర్చు. ఖాళీ tank తో ఏదో ఒక station నుండి మొదలై, **మొత్తం circuit** ఒకసారి తిరిగి రాగలిగే starting station **index** ని return చెయ్యి. సాధ్యం కాకపోతే `-1`. Solution ఉంటే అది **unique**.
- **Key constraints:** Circular. ఒకే valid start (ఉంటే).
- **ఉదాహరణ:**

Input: `gas = [1,2,3,4,5]`, `cost = [3,4,5,1,2]`
 Output: 3 (station 3 నుండి మొదలుపెడితే circuit పూర్తవుతుంది)

- **ఎలా ఆలోచించాలి:**

Naive: ప్రతి station ని start గా try చేసి, circuit simulate $\rightarrow O(n^2)$.

రెండు insights:

1. **మొత్తం `gas` $\geq$ మొత్తం `cost` అయితేనే** solution ఉంది (లేకపోతే అసలు పెట్రోల్ చాలదు) $\rightarrow -1$.
`sum(gas[i] - cost[i])  $\geq$  0` అయితే solution guaranteed.
2. **Start ని ఎలా కనుక్కోవాలి?** ఒక station `A` నుండి మొదలుపెట్టి, station `B` కి వెళ్ళేసరికి tank negative అయిందనుకో. అప్పుడు – `A` నుండి `B` మధ్య **ఏ station నుండి మొదలుపెట్టినా** `B` దాటలేం! ఎందుకంటే `A` నుండి మొదలుపెడితే ఆ మధ్య stations దగ్గర tank ≥ 0 ఉండేది; అలాంటప్పుడు వాటి నుండి మొదలుపెడితే tank ఇంకా తక్కువ (initial surplus లేదు). కాబట్టి **తర్వాతి station (`B+1`) నుండి fresh start** try చెయ్యి.

ఇది greedy: ఒక్క pass లో running tank maintain చేస్తూ, negative అయినప్పుడల్లా start ని i+1 కి jump చేసి tank reset. Total balance ≥ 0 అయితే, ఈ final start valid.

- **Brute Force:** ప్రతి start నుండి full circuit simulate $\rightarrow O(n^2)$.
- **Optimal Approach: Greedy one-pass – total balance + reset start on deficit.** $O(n)$ time, $O(1)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} gas
 * @param {number[]} cost
 * @return {number} - starting index, లేదా -1
 */
var canCompleteCircuit = function (gas, cost) {
    let total = 0; // మొత్తం surplus (solution ఉందా అని తేల్చడానికి)
    let tank = 0; // ప్రస్తుత start నుండి ఇప్పటిదాకా tank
    let start = 0; // candidate starting station

    for (let i = 0; i < gas.length; i++) {
        const diff = gas[i] - cost[i]; // ఈ station దగ్గర net పెట్టేల్
        total += diff;
        tank += diff;

        if (tank < 0) {
            // ఇక్కడ చిక్కుకున్నాం  $\rightarrow$  start..i మధ్య ఏదీ పని చెయ్యదు
            start = i + 1; // తర్వాతి station నుండి fresh start
            tank = 0; // tank reset
        }
    }

    return total >= 0 ? start : -1; // మొత్తం surplus ఉంటేనే start valid
};
```

- **Dry Run:** gas=[1,2,3,4,5], cost=[3,4,5,1,2] .

i	diff=gas-cost	total	tank (after)	tank<0?	start
0	1-3=-2	-2	-2	yes	1 (reset tank=0)
1	2-4=-2	-4	-2	yes	2 (reset tank=0)
2	3-5=-2	-6	-2	yes	3 (reset tank=0)
3	4-1=3	-3	3	no	3
4	5-2=3	0	6	no	3

total=0 ≥ 0 $\rightarrow$ return start=3. ✓

- **Complexity:** Time $O(n)$ – ఒక్క pass. Space $O(1)$.

- **గుర్తుంచుకోవాల్సినది:** "**Feasibility = total balance ≥ 0 ; start = deficit తర్వాతి station.**" "ఒక range లో ఎక్కడైనా fail అయితే, ఆ range మొత్తాన్ని skip చెయ్యి" అనే greedy elimination – circular/prefix-sum problems లో powerful. రెండు వేరు variables (total for existence, tank for current run) పెట్టడం key.
- **సాధారణ తప్పులు:**
 - `total` ని విడిగా track చెయ్యకపోవడం – tank reset అవుతుంది కాబట్టి "solution ఉందా" అని tank చెప్పలేదు; total చెప్పింది.
 - `tank < 0` అయినప్పుడు `start = i` (`i+1` కాదు) పెట్టడం – ప్రస్తుత `i` కూడా fail కారణం, దాటాలి.
 - `tank = 0` reset మర్చిపోవడం.

15. Candy (LeetCode #135) – Hard

- **సమస్య:** `n` పిల్లలు వరుసలో, ప్రతి ఒక్కరికీ `ratings[i]` . Candies పంచాలి:
 - ప్రతి పిల్లవాడికీ **కనీసం 1** candy.
 - **ఎక్కువ rating** ఉన్న పిల్లవాడికీ, తన **వక్క** (adjacent) పిల్లవాడి కంటే ఎక్కువ candies.
 - కావాల్సిన **కనిష్ఠ total** candies ఎంత?
- **Key constraints:** రెండు వైపుల neighbors (ఎడమ + కుడి) rule. Minimum total.
- **ఉదాహరణ:**

```
Input: ratings = [1,0,2]
Output: 5    ([2,1,2] → 2+1+2=5)
```

- **ఎలా ఆలోచించాలి:**

రెండు constraints ఉన్నాయి: "ఎడమ neighbor కంటే ఎక్కువ rating అయితే ఎక్కువ candy" మరియు "కుడి neighbor కంటే ఎక్కువ rating అయితే ఎక్కువ candy." ఈ రెండింటినీ **ఒకేసారి** ఒక్క pass లో satisfy చెయ్యడం కష్టం – ఒకటి fix చేస్తే ఇంకోటి break అవుతుంది.

ఏం గమనించాలి? (Two-pass – విడదీయి) రెండు rules ని విడిగా handle చెయ్యి:

1. **ఎడమ→కుడి pass:** అందరికీ 1 candy తో మొదలు. `ratings[i] > ratings[i-1]` అయితే `candies[i] = candies[i-1] + 1` . ఇది **ఎడమ** neighbor rule ని satisfy చేస్తుంది.
2. **కుడి→ఎడమ pass:** `ratings[i] > ratings[i+1]` అయితే `candies[i] = max(candies[i], candies[i+1] + 1)` . **కుడి** neighbor rule. `max` ముఖ్యం – మొదటి pass లో సంపాదించిన value ని పాడు చెయ్యకూడదు.

రెండు passes తర్వాత రెండు rules ఏకకాలంలో satisfy అవుతాయి (max వల్ల). Total = sum. ఇదే classic **two-pass** pattern.

- **Brute Force:** ఏదైనా పిల్లవాడి candy మారితే neighbors ని మళ్ళీ adjust చేస్తూ convergeయ్యేదాకా repeat $\rightarrow O(n^2)$ worst case.
- **Optimal Approach: Two-pass (left-to-right + right-to-left with max).** $O(n)$ time, $O(n)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} ratings
 * @return {number} - కనిష్ట total candies
 */
var candy = function (ratings) {
  const n = ratings.length;
  const candies = new Array(n).fill(1); // అందరికీ కనీసం 1

  // Pass 1 (ఎడమ→కుడి): ఎడమ neighbor కంటే ఎక్కువ rating  $\rightarrow$  ఎక్కువ candy
  for (let i = 1; i < n; i++) {
    if (ratings[i] > ratings[i - 1]) {
      candies[i] = candies[i - 1] + 1;
    }
  }

  // Pass 2 (కుడి→ఎడమ): కుడి neighbor కంటే ఎక్కువ rating  $\rightarrow$  ఎక్కువ candy
  // max తో pass 1 ఫలితం కాపాడు
  for (let i = n - 2; i >= 0; i--) {
    if (ratings[i] > ratings[i + 1]) {
      candies[i] = Math.max(candies[i], candies[i + 1] + 1);
    }
  }

  return candies.reduce((sum, c) => sum + c, 0);
};
```

- **Dry Run:** ratings=[1,0,2] . Start candies=[1,1,1] .

Pass 1 (ఎడమ→కుడి): i=1: 0>1? no $\rightarrow$ [1,1,1]. i=2: 2>0? yes $\rightarrow$ candies[2]=candies[1]+1=2 $\rightarrow$ [1,1,2] .

Pass 2 (కుడి→ఎడమ): i=1: ratings[1]=0 > ratings[2]=2? no $\rightarrow$ [1,1,2]. i=0: ratings[0]=1 > ratings[1]=0? yes $\rightarrow$ candies[0]=max(1, candies[1]+1=2)=2 $\rightarrow$ [2,1,2] .

Sum = 2+1+2 = 5 . 

- **Complexity:** Time $O(n)$ – two passes. Space $O(n)$ – candies array.
- **గుర్తుంచుకోవాల్సినది:** "రెండు వైపుల neighbors constraint $\rightarrow$ two-pass, రెండో pass లో **Math.max** తో merge." ఒక్క pass లో satisfy కాని local constraints ని directions గా విడదీయడం – Trapping Rain Water (#42), Product Except Self (#238) లో కూడా అదే ఆత్మ. **Math.max** మర్చిపోతే మొత్తం తప్పు.
- **సాధారణ తప్పులు:**

- Pass 2 లో `Math.max` వాడకుండా నేరుగా `candies[i] = candies[i+1] + 1` – pass 1 లో వచ్చిన valid ఎక్కువ value ని పాడు చేస్తుంది.
- `ratings[i] == ratings[i-1]` (equal) కి ఎక్కువ candy ఇవ్వడం – "ఎక్కువ rating" మాత్రమే, equal కి కాదు (strictly greater).
- Passes direction తప్పు (రెండూ ఒకే direction) – రెండూ opposite కావాలి.

16. Trapping Rain Water (LeetCode #42) – Hard

- **సమస్య:** `height[i]` = ఒక bar యొక్క ఎత్తు (వెడల్పు 1). వర్షం కురిసినప్పుడు ఈ elevation map ఎంత నీటిని నిల్వ చేస్తుందో లెక్కించు.
- **Key constraints:** ప్రతి index దగ్గర నిల్వ నీరు ఆ index కి ఎడమవైపు గరిష్ట ఎత్తు & కుడివైపు గరిష్ట ఎత్తు పై ఆధారపడుతుంది.
- **ఉదాహరణ:**

Input: `height = [0,1,0,2,1,0,1,3,2,1,2,1]`
Output: 6

- **ఎలా ఆలోచించాలి:**

ఒక్క index `i` దగ్గర ఎంత నీరు నిల్వ అవుతుంది? దాని ఎడమవైపు ఎత్తైన గోడ `leftMax`, కుడివైపు ఎత్తైన గోడ `rightMax`. నీటి మట్టం = `min(leftMax, rightMax)` (తక్కువ గోడ నుండి పొంగిపోతుంది). ఆ index దగ్గర నీరు = `min(leftMax, rightMax) - height[i]` (బార్ ఎత్తు తీసేసి). ఇది negative కాకూడదు.

Naive: ప్రతి `i` కి ఎడమ, కుడి max ని scan $\rightarrow O(n^2)$. లేదా precompute `leftMax[]`, `rightMax[]` arrays (two-pass) $\rightarrow O(n)$ time, $O(n)$ space.

ఏం గమనించాలి? (Two pointers, $O(1)$ space) రెండు pointers `left`, `right` రెండు చివరల నుండి. `leftMax`, `rightMax` running గా. కీలక observation: `height[left] < height[right]` అయితే – `left` దగ్గర నీరు **leftMax మీదే** ఆధారపడుతుంది (కుడివైపు కనీసం `height[right]` ఉంది, అది `leftMax` కంటే ఎక్కువ కాబట్టి, min ఎప్పుడూ `leftMax`). కాబట్టి `left` ని safely process చేసి ముందుకు జరపొచ్చు – `rightMax` కచ్చితంగా తెలియకపోయినా! తక్కువ వైపు నుండి process చెయ్యడం వల్ల ఆ side max నిర్ణయాత్మకం.

- **Brute Force:** ప్రతి `i` కి ఎడమ/కుడి max scan $\rightarrow O(n^2)$. లేదా precomputed arrays $\rightarrow O(n)$ time, $O(n)$ space.
- **Optimal Approach: Two pointers converging + running leftMax/rightMax.** తక్కువ ఎత్తు వైపు నుండి move. $O(n)$ time, $O(1)$ space.
- **Solution (JavaScript):**

```

/**
 * @param {number[]} height
 * @return {number} - నిల్వ నీరు మొత్తం
 */
var trap = function (height) {
    let left = 0;
    let right = height.length - 1;
    let leftMax = 0;
    let rightMax = 0;
    let water = 0;

    while (left < right) {
        if (height[left] < height[right]) {
            // ఎడమ వైపు తక్కువ → left దగ్గర నీరు leftMax మీద ఆధారం
            if (height[left] >= leftMax) {
                leftMax = height[left]; // కొత్త ఎడమ గోడ (ఇక్కడ నీరు 0)
            } else {
                water += leftMax - height[left]; // గోడ కంటే తక్కువ → నీరు నిల్వ
            }
            left++;
        } else {
            // కుడి వైపు తక్కువ (లేదా equal) → right దగ్గర నీరు rightMax మీద ఆధారం
            if (height[right] >= rightMax) {
                rightMax = height[right];
            } else {
                water += rightMax - height[right];
            }
            right--;
        }
    }

    return water;
};

```

- **Dry Run:** (సంక్షిప్తం) height=[0,1,0,2,1,0,1,3,2,1,2,1] . left=0,right=11.

కొన్ని steps: $h[0]=0 < h[11]=1 \rightarrow \text{leftMax}=0$ ($0 \geq 0$), left→1. $h[1]=1 < h[11]=1$? no (equal, else branch) → $h[11]=1 \geq \text{rightMax}=0 \rightarrow \text{rightMax}=1$, right→10. $h[1]=1 < h[10]=2 \rightarrow 1 \geq \text{leftMax}=0 \rightarrow \text{leftMax}=1$, left→2. $h[2]=0 < h[10]=2 \rightarrow 0 < \text{leftMax}=1 \rightarrow \text{water} += 1-0=1$, left→3. $h[3]=2 < 2$? no → $h[10]=2 \geq \text{rightMax}=1 \rightarrow \text{rightMax}=2$, right→9. $h[3]=2 < h[9]=1$? no → $1 < \text{rightMax}=2 \rightarrow \text{water} += 2-1=1$ (total 2), right→8. $h[3]=2 < h[8]=2$? no → $2 < \text{rightMax}=2$? no, $2 \geq 2 \rightarrow \text{rightMax}=2$, right→7. $h[3]=2 < h[7]=3 \rightarrow 2 \geq \text{leftMax}=1 \rightarrow \text{leftMax}=2$, left→4. $h[4]=1 < 3 \rightarrow 1 < \text{leftMax}=2 \rightarrow \text{water} += 1$ (total 3), left→5. $h[5]=0 < 3 \rightarrow \text{water} += 2$ (total 5), left→6. $h[6]=1 < 3 \rightarrow \text{water} += 1$ (total 6), left→7. left=7, right=7 → stop.

Total 6. 

- **Complexity:** Time $O(n)$ – ప్రతి pointer ఒక్కసారి కదులుతుంది. Space $O(1)$.
- గుర్తుంచుకోవాల్సినది: "ప్రతి cell రెండు వైపుల max మీద ఆధారపడితే → two pointers, తక్కువ వైపు నుండి process." "తక్కువ వైపు max నిర్ణయాత్మకం" అనే insight చాలా container/water problems

(Container With Most Water #11 కూడా) కీ core. Two-pass array version కూడా తెలుసుకో – అది explain చెయ్యడం సులభం.

- **సాధారణ తప్పులు:**

- `min(leftMax, rightMax) - height[i]` logic ని ఏ pointer move చెయ్యాలో గందరగోళం.
Rule: **తక్కువ height ఉన్న వైపు** process & move.
- `>=` vs `>` – `height[left] >= leftMax` అయితే leftMax update (నీరు 0), లేకపోతే నీరు కూడు.
తప్పు comparison → negative water.
- Precompute version లో leftMax/rightMax boundaries (`leftMax[0]=height[0]`) తప్పు set చెయ్యడం.

17. Roman to Integer (LeetCode #13) – Easy

- **సమస్య:** Roman numeral string ని integer గా మార్చు. Symbols: `I=1, V=5, X=10, L=50, C=100, D=500, M=1000`. సాధారణంగా పెద్దది → చిన్నది క్రమంలో కూడి. కానీ **subtractive** cases: `IV=4, IX=9, XL=40, XC=90, CD=400, CM=900` (చిన్నది పెద్దదికి **ముందు** వస్తే తీసివేత).
- **Key constraints:** Valid Roman input (1..3999).
- **ఉదాహరణ:**

```
Input: s = "MCMXCIV"
Output: 1994 (M=1000, CM=900, XC=90, IV=4)
```

- **ఎలా ఆలోచించాలి:**

మామూలుగా అయితే అన్ని symbols విలువలు కూడేయాలి. కానీ subtractive notation trap: `IV = 4, VI = 6`. తేడా – **చిన్న symbol పెద్దదానికి ముందు వస్తే తీసివేయాలి.**

ఏం గమనించాలి? ప్రతి symbol ని దాని **తర్వాతి symbol** తో పోల్చు. `current < next` అయితే – ఇది subtractive (`current` ని తీసివేయి). లేకపోతే కూడు. ఉదా `IV`: `I(1) < V(5) → I` ని `-1`. ఒక్క left-to-right pass సరిపోతుంది.

Analogy: బిల్లు చదువుతున్నట్టు. చిన్న నోటు పెద్ద నోటు **ముందు** కనిపిస్తే – "ఇది తగ్గింపు" అని అర్థం (discount coupon పెద్ద amount ముందు).

- **Brute Force:** అన్ని 2-character subtractive combos ని ముందు వెతికి replace చేసి తర్వాత కూడడం – messy, error-prone. అవసరం లేదు.
- **Optimal Approach: Single pass,** `current < next` అయితే **subtract else add**. $O(n)$ time, $O(1)$ space.
- **Solution (JavaScript):**

```

/**
 * @param {string} s
 * @return {number}
 */
var romanToInt = function (s) {
  const value = { I: 1, V: 5, X: 10, L: 50, C: 100, D: 500, M: 1000 };
  let total = 0;

  for (let i = 0; i < s.length; i++) {
    const current = value[s[i]];
    const next = value[s[i + 1]]; // చివరి character దగ్గర undefined → subtract branch skip

    if (next !== undefined && current < next) {
      total -= current; // subtractive (IV, IX, ...)
    } else {
      total += current;
    }
  }

  return total;
};

```

- **Dry Run:** `s="MCMXCIV"` .

i	s[i]	current	next	current<next?	action	total
0	M	1000	100(C)	no	+1000	1000
1	C	100	1000(M)	yes	-100	900
2	M	1000	10(X)	no	+1000	1900
3	X	10	100(C)	yes	-10	1890
4	C	100	1(I)	no	+100	1990
5	I	1	5(V)	yes	-1	1989
6	V	5	undefined	no	+5	1994

Return `1994` . 

- **Complexity:** Time `O(n)` – ఒక్క pass. Space `O(1)` (constant-size map).
- **గుర్తుంచుకోవాల్సినది:** "పక్క element తో compare చేసి decision (subtract/add)" – **lookahead pattern**. Roman కి రహస్యం: `current < next` అయితే subtract. String ని hash map తో decode చెయ్యడం – parsing problems కి common.
- **సాధారణ తప్పులు:**
 - `next` undefined check మర్చిపోతే `current < undefined = false` (JS quirk) – ఇక్కడ అదృష్టవశాత్తు add branch కి వెళ్తుంది, కానీ explicit check clearer & safe.

- Subtractive ని += తో handle చెయ్యడం (సరైన logic కాదు).
- Symbol values hardcode తప్పుగా (L=50, C=100 తిక్మక).

18. Integer to Roman (LeetCode #12) – Medium

- **సమస్య:** Integer (1..3999) ని Roman numeral string గా మార్చు. Subtractive forms కూడా వాడాలి: 4=IV, 9=IX, 40=XL, 90=XC, 400=CD, 900=CM.
- **Key constraints:** గరిష్టంగా చిన్న (canonical) representation. Range 1..3999.
- **ఉదాహరణ:**

```
Input: num = 1994
Output: "MCMXCIV" (1000=M, 900=CM, 90=XC, 4=IV)
```

- **ఎలా ఆలోచించాలి:**

Roman లో ప్రతి "అంకె స్థానం" (thousands, hundreds, tens, units) కి కొన్ని fixed symbols. అంటే number నుండి పెద్ద denomination లు వరుసగా తీసేస్తూ, ఆ symbol ని append చేస్తూ వెళ్తే సరిపోతుంది – currency change ఇచ్చినట్టు (పెద్ద నోట్ల నుండి).

ఏం గమనించాలి? (Greedy తో subtractive forms ని values గా చేర్చడం) అసలు trick: 4, 9, 40, 90, 400, 900 లాంటి subtractive cases ని కూడా **విలువల list** లో పెట్టేయి. అప్పుడు greedy గా "ప్రస్తుత విలువ num కంటే తక్కువ/సమానం అయినంతవరకు symbol append చేసి subtract" చేస్తే – subtractive forms automatic గా handle అవుతాయి. **values** ని descending order లో ఉంచడం key.

- **Brute Force:** ప్రతి digit (thousands/hundreds/tens/units) విడిగా handle చేసి, 4 lookup tables. పని చేస్తుంది కానీ code పొడవు. Greedy list చిన్నది & elegant.
- **Optimal Approach: Greedy descending – subtractive forms ని values array లో చేర్చి, ప్రతి value ని fits అయినంతవరకు subtract & append.** O(1) time (bounded ≤3999), O(1) space.
- **Solution (JavaScript):**

```
/**
 * @param {number} num (1..3999)
 * @return {string}
 */
var intToRoman = function (num) {
    // subtractive forms (900,400,90,40,9,4) కూడా చేర్చాలి, descending order
    const values = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1];
    const symbols = ['M', 'CM', 'D', 'CD', 'C', 'XC', 'L', 'XL', 'X', 'IX', 'V', 'IV', 'I'];

    let result = '';
    for (let i = 0; i < values.length; i++) {
        // ప్రస్తుత denomination ఎన్నిసార్లు fits అవుతుందో అన్నిసార్లు append
    }
```

```

while (num >= values[i]) {
    result += symbols[i];
    num -= values[i];
}
if (num == 0) break; // ఖాళీ అయితే ఆగు
}

return result;
};

```

• **Dry Run:** num=1994 .

i	values[i]	symbols[i]	ఎన్నిసార్లు	result	num (after)
0	1000	M	1	"M"	994
1	900	CM	1	"MCM"	94
2	500	D	0	"MCM"	94
3	400	CD	0	"MCM"	94
4	100	C	0	"MCM"	94
5	90	XC	1	"MCMXC"	4
...	50..9		0	"MCMXC"	4
11	4	IV	1	"MCMXCIV"	0

Return "MCMXCIV" . 

- **Complexity:** Time $O(1)$ – values 13 fixed, num ≤ 3999 కాబట్టి iterations bounded. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది: "Greedy denomination decomposition – special/subtractive cases ని denominations list లోనే చేర్చు."** ఇది coin-change-లాంటి greedy (Roman denominations canonical కాబట్టి greedy optimal). "6, 9, 40 లాంటి exceptions" ని data లో encode చేస్తే code branch-free & clean.
- **సాధారణ తప్పులు:**
 - Subtractive forms (CM, XL, IV) ని list లో మర్చిపోవడం → IIII లాంటి తప్పు output.
 - values descending order లో లేకపోవడం → greedy fail.
 - while బదులు if వాడడం → ఒకే denomination బహుసార్లు (III) handle కాదు.

19. Length of Last Word (LeetCode #58) – Easy

- **సమస్య:** ఒక string `s` (words + spaces) ఇస్తారు. **చివరి word** యొక్క length ని return చెయ్యి. Word = space లేని characters. Trailing spaces ఉండవచ్చు.
- **Key constraints:** కనీసం ఒక word guaranteed. Trailing/multiple spaces edge cases.
- **ఉదాహరణ:**

```
Input: s = " fly me to the moon "  
Output: 4 (చివరి word "moon", length 4)
```

- **ఎలా ఆలోచించాలి:**

Easy approach: `s.trim().split(/\s+/)`, చివరి element length. పని చేస్తుంది. కానీ ఇది మొత్తం string ని process చేస్తుంది (extra array కూడా).

ఏం గమనించాలి? (వెనుక నుండి scan) మనకి కావాల్సింది చివరి word మాత్రమే. కాబట్టి **వెనుక నుండి** (end నుండి) నడు:

1. మొదట trailing spaces ని skip చెయ్యి (చివర్లో ఖాళీలుంటే).
2. తర్వాత space వచ్చేదాకా అక్షరాలు లెక్కించు – అదే చివరి word length.

మొత్తం string touch చేయక్కర్లేదు, extra array అవసరం లేదు. String parsing pattern యొక్క neat application.

- **Brute Force:** `split` + `trim` → చివరి element. Time $O(n)$, Space $O(n)$ (array of words). సరిపోతుంది కానీ extra space.
- **Optimal Approach: Backward scan – trailing spaces skip, తర్వాత చివరి word count.** $O(n)$ worst case time, $O(1)$ space.
- **Solution (JavaScript):**

```
/**  
 * @param {string} s  
 * @return {number}  
 */  
var lengthOfLastWord = function (s) {  
    let i = s.length - 1;  
  
    // 1) trailing spaces skip  
    while (i >= 0 && s[i] === ' ') {  
        i--;  
    }  
  
    // 2) చివరి word length లెక్కించు (space లేదా start వచ్చేదాకా)  
    let length = 0;  
    while (i >= 0 && s[i] !== ' ') {  
        length++;  
        i--;  
    }  
}
```

```

    }

    return length;
};

```

- **Dry Run:** `s=" fly me to the moon "` (చివర్లో 2 spaces).

Step 1: `i` = last index. `s[i]=' '` → skip trailing spaces. "moon" లోని `n` దగ్గర ఆగుతుంది. Step 2: `n, o, o, m` – 4 అక్షరాలు లెక్కించి, దాని ముందు space వచ్చాక ఆగుతుంది. `length=4`.

Return `4`. ✓

- **Complexity:** Time `O(n)` worst case (అన్నీ spaces అయితే), సాధారణంగా చివరి word + trailing spaces మాత్రమే. Space `O(1)`.
- **గుర్తుంచుకోవాల్సినది:** "చివరిది మాత్రమే కావాలంటే వెనుక నుండి scan – మొత్తం process అవసరం లేదు." Two-phase scan (skip delimiters → collect token) అనేది String parsing యొక్క fundamental idiom. Built-in `split` తెలిసినా, "without built-ins" అడిగితే ఇది రెడీగా ఉంచుకో.
- **సాధారణ తప్పులు:**
 - Trailing spaces skip చెయ్యకపోవడం → "moon " లో spaces ని కూడా count లేదా length 0.
 - `s.split(' ')` (single space) వాడి multiple spaces వల్ల ఖాళీ strings వచ్చి, చివరిది "" – regex `\s+` లేదా filter అవసరం.
 - `i >= 0` bound మర్చిపోతే `s[-1]` = undefined తో infinite loop.

20. Longest Common Prefix (LeetCode #14) – Easy

- **సమస్య:** Strings array `strs` ఇస్తారు. అన్నిటికీ common అయిన **longest prefix** (మొదటి నుండి కలిసే భాగం) ని return చెయ్యి. లేకపోతే "".
- **Key constraints:** ఖాళీ array, ఖాళీ strings edge cases.
- **ఉదాహరణ:**

```

Input:  strs = ["flower","flow","flight"]
Output: "fl"

```

- **ఎలా ఆలోచించాలి:**

Common prefix అంటే – అన్ని strings లో **అదే position లో అదే character** ఉన్నంతవరకు. మొదటి mismatch వచ్చిన చోట prefix ఆగిపోతుంది.

ఏం గమనించాలి? (Vertical scanning) మొదటి string ని reference గా తీసుకో. దాని ప్రతి character `strs[0][i]` ని, మిగతా అన్ని strings లోని అదే position తో compare చెయ్యి (**column by column** –

vertical). ఏ string లోనైనా mismatch వచ్చినా, లేదా ఆ string అయిపోయినా – అక్కడితో prefix `strs[0].slice(0, i)` . అన్ని columns match అయితే మొత్తం `strs[0]` prefix.

ఎందుకు efficient? Common prefix అతి చిన్న string కంటే పొడవుగా ఉండదు, పైగా mismatch వచ్చిన వెంటనే ఆగుతాం.

- **Brute Force:** ఒక candidate prefix (`strs[0]`) తీసుకుని, ప్రతి string కి match అయ్యేదాకా చివరి character తీస్తూ కుదించడం (horizontal). పని చేస్తుంది, కానీ vertical scan clearer & early-exits per column.
- **Optimal Approach: Vertical scanning – column by column, మొదటి mismatch దగ్గర ఆగు.**
 $O(S)$ time (S = అన్ని characters మొత్తం, worst case).

- **Solution (JavaScript):**

```
/**
 * @param {string[]} strs
 * @return {string}
 */
var longestCommonPrefix = function (strs) {
    if (strs.length === 0) return '';

    // మొదటి string ని reference గా, column by column
    for (let i = 0; i < strs[0].length; i++) {
        const c = strs[0][i]; // ఈ column లో expected character

        for (let j = 1; j < strs.length; j++) {
            // ఈ string అయిపోయిందా, లేదా character వేరా?
            if (i >= strs[j].length || strs[j][i] !== c) {
                return strs[0].slice(0, i); // ఇక్కడితో prefix ఆగింది
            }
        }
    }

    return strs[0]; // మొదటి string మొత్తం common prefix
};
```

- **Dry Run:** `strs=["flower","flow","flight"]` .

i	c (strs[0][i])	strs[1][i]	strs[2][i]	అన్నీ match?
0	f	f	f	yes
1	l	l	l	yes
2	o	o	i	no → return slice(0,2)="fl"

Return `"fl"` . ✓ ("flow" కి index 2 = 'o', "flight" కి index 2 = 'i' → mismatch.)

- **Complexity:** Time $O(S)$ where S = అన్ని characters మొత్తం (worst case అన్నీ match). నిజానికి $O(\minLen \times \text{numStrings})$ upper bound, mismatch దగ్గర early exit. Space $O(1)$ (slice తప్ప).

- గుర్తుంచుకోవాల్సినది: "అన్ని strings కి common ఏదైనా = column-wise (vertical) compare, మొదటి mismatch దగ్గర ఆగు." ఒక reference string వాడి మిగతా వాటితో పోల్చడం – comparison-across-collection problems కి common. Early exit efficiency ఇస్తుంది.
- సాధారణ తప్పులు:
 - `i >= strs[j].length` bound check మర్చిపోవడం → చిన్న string ("flow", 4 అక్షరాలు) దాటాక `strs[j][i]` = undefined, తప్పు match.
 - ఖాళీ array / ఖాళీ string ("") edge cases handle చెయ్యకపోవడం.
 - `sort` చేసి first & last మాత్రమే compare చేసే trick – pattern గా correct కానీ $O(n \log n)$ sort అనవసరం ఇక్కడ.

21. Reverse Words in a String (LeetCode #151) – Medium

- సమస్య: ఒక string `s` ఇస్తారు. అందులోని words క్రమాన్ని తలకిందులు చేసి return చెయ్యి. Output లో words మధ్య ఒకే space, leading/trailing spaces ఉండకూడదు. Input లో multiple spaces ఉండవచ్చు.
- **Key constraints:** Extra spaces clean చెయ్యాలి. Words లోపల reverse కాదు – words order reverse.
- ఉదాహరణ:

```
Input: s = " the sky is blue "
Output: "blue is sky the"
```

- ఎలా ఆలోచించాలి:

కావాల్సింది: words ని విడదీసి, extra spaces తీసేసి, order తిప్పి, ఒకే space తో కలపడం.

Clean approach: `s.trim()` (leading/trailing spaces తీసేయి) → `.split(/\s+/)` (ఒకటి లేదా అంతకంటే ఎక్కువ spaces మధ్య విడదీయి, ఖాళీ tokens రావు) → `.reverse()` → `.join(' ')` (ఒకే space తో కలుపు). చాలా readable.

ఏం గమనించాలి? (manual, $O(1)$ extra space in languages with mutable strings) JS strings immutable కాబట్టి pure $O(1)$ space కుదరదు, కానీ interview follow-up "in-place" (mutable char array) అడిగితే: మొత్తం string reverse → ప్రతి word ని విడిగా reverse → spaces clean. ఇది Rotate Array (#189) reversal idea కి string version. JS లో practical గా split/reverse/join సరిపోతుంది; concept తెలిస్తే చాలు.

- **Brute Force:** Character by character parse చేసి words ని stack లో push, తర్వాత pop చేస్తూ build. పని చేస్తుంది, కానీ split/reverse/join concise.

- **Optimal Approach: trim + split(/s+/) + reverse + join.** $O(n)$ time, $O(n)$ space (words array – strings immutable కాబట్టి తప్పదు).

- **Solution (JavaScript):**

```
/**
 * @param {string} s
 * @return {string}
 */
var reverseWords = function (s) {
    return s
        .trim()           // leading/trailing spaces తీసేయి
        .split(/s+/)      // ఒకటి+ spaces మీద విడదీయి (ఖాళీ tokens రావు)
        .reverse()        // words order తలకిందులు
        .join(' ');       // ఒకే space తో కలుపు
};
```

Manual version (built-ins లేకుండా – follow-up కి):

```
var reverseWordsManual = function (s) {
    const words = [];
    let i = 0;
    const n = s.length;

    while (i < n) {
        while (i < n && s[i] === ' ') i++;          // spaces skip
        let start = i;
        while (i < n && s[i] !== ' ') i++;          // ఒక word చదువు
        if (i > start) words.push(s.slice(start, i));
    }

    // words ని వెనుక నుండి కలుపు
    let result = '';
    for (let k = words.length - 1; k >= 0; k--) {
        result += words[k];
        if (k > 0) result += ' ';
    }
    return result;
};
```

- **Dry Run:** `s=" the sky is blue "`.

- `trim()` → `"the sky is blue"`.
- `split(/s+/)` → `["the","sky","is","blue"]` (multiple spaces ఒకే separator గా).
- `reverse()` → `["blue","is","sky","the"]`.
- `join(' ')` → `"blue is sky the"`. ✓

- **Complexity:** Time $O(n)$. Space $O(n)$ – words array & output (JS strings immutable).

- **గుర్తుంచుకోవాల్సినది: "Words reorder = split (clean) + reverse + join."** `split(/s+/)` (multiple spaces safe) vs `split(' ')` (ఖాళీ tokens వస్తాయి) తేడా గుర్తుంచుకో. In-place mutable version = "reverse whole + reverse each word" – Rotate Array reversal trick యొక్క cousin.

- సాధారణ తప్పులు:

- `split(' ')` వాడి multiple spaces వల్ల "" tokens రావడం (`.filter(Boolean)` అవసరం అవుతుంది).
- `trim()` మర్చిపోతే join తర్వాత leading/trailing spaces మిగలడం.
- Words లోపల characters ని reverse చెయ్యడం ("blue" → "eulb") – order మాత్రమే reverse కావాలి, letters కాదు.

22. Zigzag Conversion (LeetCode #6) – Medium

- సమస్య: String `s` ని `numRows` వరుసలలో **zigzag** pattern లో (కిందికి, తర్వాత diagonal గా పైకి, మళ్ళీ కిందికి...) రాసి, తర్వాత **row by row** చదివితే వచ్చే string ని return చెయ్యి.
- **Key constraints:** `numRows = 1` అయితే string అలాగే. Pattern ని visualize చెయ్యడం key.
- ఉదాహరణ:

```
Input: s = "PAYPALISHIRING", numRows = 3
Zigzag:
P   A   H   N
A P L S I I G
Y   I   R
Output: "PAHNAPLSIIGYIR"   (row0 + row1 + row2)
```

- ఎలా ఆలోచించాలి:

Zigzag ని literally 2D grid గా simulate చేసి coordinates లెక్కించడం సంక్లిష్టం. Simpler idea కావాలి.

ఏం గమనించాలి? ప్రతి character ఏదో ఒక **row** కి చెందుతుంది. Rows మీద మనం zigzag లా కదులుతాం: row 0 → 1 → 2 → ... → numRows-1 (కిందికి), తర్వాత numRows-2 → ... → 0 (పైకి), మళ్ళీ కిందికి. అంటే direction **పై/కింద** అంచుల దగ్గర (row 0 లేదా row numRows-1) reverse అవుతుంది.

కాబట్టి: ప్రతి row కి ఒక string buffer పెట్టు. `s` లోని ప్రతి character ని ప్రస్తుత `curRow` buffer కి append చెయ్యి, తర్వాత direction ప్రకారం `curRow` ని +1 లేదా -1 జరుపు. అంచు తాకినప్పుడు direction flip. చివర్లో అన్ని row buffers ని కలుపు. Grid coordinates అవసరం లేదు!

- **Brute Force:** వాస్తవ 2D matrix నింపి, ఖాళీలు skip చేస్తూ row by row చదవడం. $O(\text{numRows} \times n)$ space, index math కష్టం.
- **Optimal Approach: Row buffers + direction flip at edges (simulation).** $O(n)$ time, $O(n)$ space (output).
- **Solution (JavaScript):**

```
/**
 * @param {string} s
```

```

* @param {number} numRows
* @return {string}
*/
var convert = function (s, numRows) {
    if (numRows === 1) return s; // ఒక్క row → zigzag లేదు

    const rows = new Array(numRows).fill(''); // ప్రతి row కి buffer
    let curRow = 0;
    let goingDown = false; // direction: కిందికా?

    for (const char of s) {
        rows[curRow] += char; // ప్రస్తుత row కి character చేర్చు

        // అంచు (మొదటి/చివరి row) తాకితే direction flip
        if (curRow === 0 || curRow === numRows - 1) {
            goingDown = !goingDown;
        }

        curRow += goingDown ? 1 : -1; // కిందికి(+1) లేదా పైకి(-1)
    }

    return rows.join(''); // అన్ని rows ని వరుసగా కలుపు
};

```

- **Dry Run:** `s="PAYPALISHIRING", numRows=3` .

char	curRow	rows (after)	edge? flip	next curRow
P	0	r0="P"	yes → down	1
A	1	r1="A"	no	2
Y	2	r2="Y"	yes → up	1
P	1	r1="AP"	no	0
A	0	r0="PA"	yes → down	1
L	1	r1="APL"	no	2
I	2	r2="YI"	yes → up	1
...	...	...	...	...

చివర్లో `r0="PAHN", r1="APLSIIG", r2="YIR"`. Join → `"PAHNAPLSIIGYIR"` . 

- **Complexity:** Time `O(n)` – ప్రతి character ఒక్కసారే. Space `O(n)` – row buffers మొత్తం.
- **గుర్తుంచుకోవాల్సినది:** "Grid pattern ని coordinates తో కాకుండా, direction-flip simulation తో solve చెయ్యి." అంచుల దగ్గర direction reverse అయ్యే "bouncing" motion – snake/boustrophedon patterns కి common. Row buffers + join = string building లో efficient idiom.
- **సాధారణ తప్పులు:**

- `numRows === 1` edge case మర్చిపోతే `curRow` ఎప్పుడూ 0, కానీ flip logic infinite bounce లేదా wrong – early return safe.
- Direction flip ని అంచు తాకిన తర్వాత కాకుండా ముందు చెయ్యడం.
- String ని `+=` తో నేరుగా కాకుండా చాలా చోట్ల concatenate చేసి $O(n^2)$ – row buffers అయితే fine.

23. Find the Index of the First Occurrence in a String (LeetCode #28) – Easy

- **సమస్య:** రెండు strings `haystack`, `needle`. `needle` `haystack` లో మొదటిసారి ఎక్కడ (index) కనిపిస్తుందో return చెయ్యి. లేకపోతే `-1`. `needle` ఖాళీ అయితే `0`.
- **Key constraints:** Substring search. Optimal అడిగితే $O(n+m)$ (KMP).
- **ఉదాహరణ:**

```
Input: haystack = "sadbutsad", needle = "sad"
Output: 0 ("sad" మొదట index 0 దగ్గర)
```

- **ఎలా ఆలోచించాలి:**

Naive: `haystack` లో ప్రతి starting position `i` దగ్గర `needle` సరిపోతుందా అని character by character check $\rightarrow O(n \cdot m)$. చిన్న inputs కి fine, interview లో ఇది default answer.

ఏం గమనించాలి? (KMP – $O(n+m)$) Naive లో mismatch వచ్చినప్పుడు, `haystack` pointer వెనక్కి jump చేసి మళ్ళీ మొదలుపెడతాం (redundant work). KMP idea: `needle` లోని **repeated prefix structure** ముందే లెక్కిస్తే (**LPS = Longest Proper Prefix which is also Suffix** array), mismatch వచ్చినప్పుడు `haystack` వెనక్కి వెళ్ళకుండా, `needle` pointer ని మాత్రమే సరైన చోటికి jump చెయ్యవచ్చు. దీంతో `haystack` ని ఒక్కసారి scan – $O(n+m)$.

Intuition: `needle="aab"` లో mismatch వస్తే, ఇప్పటిదాకా match అయిన భాగంలో ఏ suffix మళ్ళీ `needle` prefix కి సరిపోతుందో LPS చెప్తుంది – అక్కడి నుండి continue, from scratch కాదు.

- **Brute Force:** ప్రతి window `[i..i+m-1]` ని `needle` తో compare. Time $O(n \cdot m)$, Space $O(1)$. Acceptable కానీ optimal కాదు.
- **Optimal Approach: KMP – LPS array precompute, mismatch లో needle pointer jump.** $O(n+m)$ time, $O(m)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {string} haystack
 * @param {string} needle
 * @return {number}
 */
```

```

var strStr = function (haystack, needle) {
    const m = needle.length;
    if (m === 0) return 0;

    // 1) LPS array: lps[i] = needle[0..i] లో proper prefix==suffix గరిష్ట length
    const lps = new Array(m).fill(0);
    let len = 0; // ఇప్పటిదాకా prefix match length
    let i = 1;
    while (i < m) {
        if (needle[i] === needle[len]) {
            lps[i++] = ++len;
        } else if (len > 0) {
            len = lps[len - 1]; // వెనక్కి fallback (from scratch కాదు)
        } else {
            lps[i++] = 0;
        }
    }

    // 2) Search: haystack ని ఒక్కసారి scan
    let j = 0; // needle లో ప్రస్తుత match position
    for (let k = 0; k < haystack.length; k++) {
        while (j > 0 && haystack[k] !== needle[j]) {
            j = lps[j - 1]; // mismatch → needle pointer ని fallback
        }
        if (haystack[k] === needle[j]) j++;
        if (j === m) return k - m + 1; // పూర్తి match - start index
    }

    return -1;
};

```

- **Dry Run:** haystack="sadbutsad", needle="sad" .

LPS for "sad": ఏ prefix=suffix repeat లేదు → lps=[0,0,0] .

Search: k=0 's' matches needle[0] → j=1. k=1 'a' matches needle[1] → j=2. k=2 'd' matches needle[2] → j=3. j===3(m) → return 2-3+1 = 0 . ✓

- **Complexity:** Time $O(n+m)$ – LPS build $O(m)$ + search $O(n)$ (haystack pointer వెనక్కి వెళ్ళదు).
Space $O(m)$ – LPS array. (Brute: $O(n \cdot m)$ time, $O(1)$ space.)

- **గుర్తుంచుకోవాల్సినది: "Substring search optimal = KMP; core idea = mismatch లో pattern యొక్క prefix-suffix (LPS) వాడి haystack ని వెనక్కి తనికెళ్ళకపోవడం."** Interview లో ముందు brute force చెప్పి, తర్వాత "O(n+m) కావాలంటే KMP" అని LPS explain చెయ్యి. LPS concept string problems లో మళ్ళీ వస్తుంది (repeated substring patterns).

- **సాధారణ తప్పులు:**

- ఖాళీ needle కి 0 return మర్చిపోవడం.
- LPS build లో len = lps[len-1] fallback బదులు len-- చెయ్యడం (తప్పు, $O(m^2)$).
- KMP గుర్తులేకపోతే brute force ఇచ్చినా OK – కానీ off-by-one (i + m <= n bound) జాగ్రత్త.

24. Text Justification (LeetCode #68) – Hard

- **సమస్య:** Words array `words` మరియు `maxWidth` ఇస్తారు. Text ని **fully justified** (ఎడమ + కుడి రెండు అంచులూ సమం) format చెయ్యి:
 - ప్రతి line లో సాధ్యమైనన్ని words (greedy గా). Words మధ్య కనీసం ఒక space.
 - Extra spaces ని gaps మధ్య **సమానంగా** పంచు; సమానంగా రాకపోతే **ఎడమవైపు gaps కి ఎక్కువ**.
 - **చివరి line** మరియు **ఒక్క word ఉన్న line** – **left-justified** (words మధ్య ఒకే space, మిగతా trailing spaces కుడివైపు).
 - ప్రతి line సరిగ్గా `maxWidth` characters.
- **Key constraints:** ప్రతి word length $\leq$ maxWidth. చాలా edge cases – ఇదే దీన్ని Hard చేస్తుంది.
- **ఉదాహరణ:**

```
Input: words = ["This","is","an","example","of","text","justification."], maxWidth = 16
Output:
[
  "This    is    an",
  "example  of text",
  "justification. "
]
```

- **ఎలా ఆలోచించాలి:**

ఇది algorithm కంటే **careful simulation + edge cases** problem. మూడు దశలుగా విడదీయి:

1. **ఏ words ఏ line లో?** (greedy packing) – ఒక line లో words[i..j-1] పెడితే కనీస width = (word lengths sum) + (gaps = word count - 1). ఇంకో word చేర్చితే `maxWidth` దాటుతుందా అని check చేస్తూ ఎన్ని fit అవుతాయో అన్ని తీసుకో.
2. **Spaces ఎలా పంచాలి?** Line లో total spaces = `maxWidth - (word lengths sum)`. Gaps = `wordCount - 1`. `spacePer = floor(total/gaps)`, మిగిలిన `extra = total % gaps` ని **మొదటి extra gaps** కి ఒక్కొక్కటి అదనంగా (ఎడమవైపు ఎక్కువ).
3. **Special cases:** చివరి line, లేదా ఒక్క word ఉన్న line → left-justify (words ఒకే space, చివర్లో trailing spaces తో `maxWidth` కి pad).

ఏం గమనించాలి? "ఎడమవైపు ఎక్కువ" rule ని `k - i < extra` తో handle చెయ్యడం, gaps=0 (single word) అయితే division by zero తప్పించడానికి special branch – ఇవి రెండూ కీలక traps.

- **Brute Force:** ఒకటే direct approach ఉంది – greedy packing + careful spacing. "Brute" అంటే ప్రతి spacing combination try చెయ్యడం అనవసరం; greedy packing optimal (fewest lines).
- **Optimal Approach: Greedy line packing + per-line space distribution (simulation), last/single-word line special-cased.** O(total characters) time.
- **Solution (JavaScript):**

```

/**
 * @param {string[]} words
 * @param {number} maxWidth
 * @return {string[]}
 */
var fullJustify = function (words, maxWidth) {
    const res = [];
    const n = words.length;
    let i = 0;

    while (i < n) {
        // 1) ఈ line లో ఎన్ని words fit అవుతాయో కనుక్కో (words[i..j-1])
        let j = i;
        let lineLen = 0; // ప్రస్తుత line లోని word characters మొత్తం (spaces కాదు)
        // words i..j మధ్య కనీస width = lineLen + words[j].len + (j - i) కనీస spaces
        while (j < n && lineLen + words[j].length + (j - i) <= maxWidth) {
            lineLen += words[j].length;
            j++;
        }

        const wordCount = j - i;
        const totalSpaces = maxWidth - lineLen; // పంచాల్సిన మొత్తం spaces
        let line = '';

        if (j === n || wordCount === 1) {
            // 3) చివరి line OR ఒక్క word → left-justify
            line = words.slice(i, j).join(' ');
            line += ' '.repeat(maxWidth - line.length); // కుడివైపు pad
        } else {
            // 2) full-justify: spaces ని gaps మధ్య పంచు
            const gaps = wordCount - 1;
            const spacePer = Math.floor(totalSpaces / gaps);
            const extra = totalSpaces % gaps; // మొదటి `extra` gaps కి +1

            for (let k = i; k < j; k++) {
                line += words[k];
                if (k < j - 1) {
                    // ఈ gap కి spaces (ఎడమవైపు gaps కి ఒకటి ఎక్కువ)
                    const spaces = spacePer + (k - i < extra ? 1 : 0);
                    line += ' '.repeat(spaces);
                }
            }
        }

        res.push(line);
        i = j; // తర్వాతి line
    }

    return res;
};

```

- **Dry Run:** words=["This","is","an","example","of","text","justification."],
maxWidth=16 .

Line 1 (i=0): packing → "This"(4), +"is"(4+2+1=7), +"an"(6+2+2=10) fit; "example" →
8+7+3=18>16 stop. wordCount=3, lineLen=8, totalSpaces=8, gaps=2, spacePer=4, extra=0. →

"This is an" (4+4+2+4+2=16). ✓

Line 2 (i=3): "example"(7), +"of"(7+2+1=10), +"text"(9+4+2=15) fit; "justification." → 13+13+3=29>16 stop. wordCount=3, lineLen=13, totalSpaces=3, gaps=2, spacePer=1, extra=1. gap0 → 1+1=2, gap1 → 1+0=1. → "example of text" (7+2+2+1+4=16). ✓

Line 3 (i=6): "justification."(13) fit; j reaches n. Last line → left-justify: "justification." + 2 trailing spaces = "justification. ". ✓

- **Complexity:** Time $O(\text{total characters})$ – ప్రతి word ఒక్కసారే process, line building linear. Space $O(\text{total})$ – output (result strings). Extra working space $O(\text{maxWidth})$ per line.
- **గుర్తుంచుకోవాల్సినది: "Hard string formatting = careful simulation, edge cases ని ముందే list చెయ్యి."** ఇక్కడ మూడు: (1) greedy pack తో + (j-i) gap accounting, (2) extra spaces ఎడమవైపుకి ($k-i < \text{extra}$), (3) last line & single-word left-justify. Interview లో edge cases ని ముందే గుర్తుచేసుకుని ఒక్కొక్కటి handle చెయ్యడం maturity చూపిస్తుంది.
- **సాధారణ తప్పులు:**
 - Packing condition లో (j - i) minimum-gaps ని మర్చిపోయి maxWidth మించడం.
 - Single word line (gaps=0) కి $\text{totalSpaces} / \text{gaps}$ → division by zero (Infinity / NaN) – special branch తప్పనిసరి.
 - Last line ని కూడా full-justify చెయ్యడం (rules ప్రకారం last line left-justify).
 - Extra spaces ని కుడివైపు gaps కి ఇవ్వడం (ఎడమవైపు కావాలి).

ముగింపు: ఈ 24 problems లో నువ్వు గమనించాల్సింది – ప్రతిదీ ఏదో ఒక **pattern** యొక్క రూపమే. Two-pointer (in-place, converging), prefix/suffix, greedy running-state, two-pass, string parsing – ఈ అయిదు నీ ఆయుధాలు. కొత్త problem వచ్చినప్పుడు "constraints ఏం చెప్తున్నాయి? (in-place? $O(n)$? sorted? both sides?)" అని అడిగి, సరైన pattern ఎంచుకో. ప్రతి problem ని కనీసం రెండుసార్లు – ఒకసారి చదివి, ఒకసారి solution చూడకుండా స్వయంగా రాసి – practice చేస్తే, interview లో ఇవి reflexes అవుతాయి. All the best! 🚀